

# **Enhancing Manipulator Control Through Image-Based Visual Servoing Techniques Using ROS2**

By

Md Masrul Khan

1931011

A thesis in partial fulfillment of the requirements for the degree of  
Bachelor of Science in Mechatronics Engineering



Department of Mechatronics Engineering  
Khulna University of Engineering & Technology  
Khulna- 9203, Bangladesh

## DECLARATION

This is to certify that the thesis work entitled “**Enhancing Manipulator Control Through Image-Based Visual Servoing Techniques using ROS2**” has been carried out by Md Masrul Khan in the Department of Mechatronics Engineering, Khulna University of Engineering & Technology, Khulna, Bangladesh. The above thesis work or any part of this work has not been submitted anywhere for the award of any degree or diploma.

### SUPERVISED BY

-----  
Dr. Md. Helal-An-Nahiyen  
Professor  
Department of Mechanical Engineering  
Khulna University of Engineering &  
Technology

### SUBMITTED BY

-----  
Md Masrul Khan  
Roll No: 1931011  
Department of Mechatronics Engineering  
Khulna University of Engineering &  
Technology

Date: 26 September 2025

## APPROVAL

This is to certify that the thesis work submitted by **Md Masrul khan**, entitled “**Enhancing Manipulator Control Through Image-Based Visual Servoing Techniques using ROS2**” has been approved by the board of examiners for the partial fulfillment of the requirements for the degree of Bachelor of Science in the Department of Mechatronics Engineering, Khulna University of Engineering & Technology, Khulna, Bangladesh in October, 2025.

## BOARD OF EXAMINERS

1. -----

Dr. Md. Helal-An Nahiyen

Supervisor

Professor

Department of Mechanical Engineering

Khulna University of Engineering & Technology

Khulna-9203, Bangladesh

2. -----

Md. Shohanur Rahman

External

Assistant Professor

Department of Mechatronics Engineering

Khulna University of Engineering & Technology

Khulna-9203, Bangladesh

## **ACKNOWLEDGEMENT**

At the very outset, all praises be to Almighty, the most gracious, the most merciful. Wherever I am standing today is due to His enormous blessings upon me. Then, I would like to take the opportunity to unveil my profound gratitude to my respected supervisor, Dr. Md. Helal-An-Nahiyen, Professor at Department of Mechanical Engineering at Khulna University of Engineering & Technology (KUET), for his unwavering support, persistent direction, and vigilant oversight throughout this work. He always gave me his valuable time when I needed it and constantly continued to encourage me in my efforts, for which I am extremely grateful.

I also want to thank my family and friends for being so understanding, encouraging, and supportive throughout the time.

September 26, 2025.

Md Masrul Khan

## **ABSTRACT**

Robot manipulators have become an integral part of modern manufacturing, healthcare, and exploration missions. Predefined models and calibration-based traditional control approaches lack in controlling precision and responsiveness in dynamic or unstructured environments. This thesis introduces a new dynamic image-based visual servoing (IBVS) approach to maximize manipulator control using real-time vision feedback. The framework combines feature detection, predictive models, and adaptive control to adjust manipulator motion in real time dynamically while guaranteeing stability and precision under varying environment conditions, noise, and latency. Experimental evaluation using ROS 2 and MoveIt Servo achieved over 85% object tracking accuracy, sub-centimeter position accuracy, and 35–50 ms per-frame processing delay. The manipulator demonstrated smooth path tracking and improved robustness against moderate occlusion and illumination variations. Overall, the method offers a scalable and low-computational solution that can be used in real-time robotic systems.

# TABLE OF CONTENTS

|                                                           |      |
|-----------------------------------------------------------|------|
| Contents                                                  |      |
| DECLARATION .....                                         | ii   |
| APPROVAL .....                                            | iii  |
| ACKNOWLEDGEMENT .....                                     | iv   |
| ABSTRACT.....                                             | v    |
| TABLE OF CONTENTS.....                                    | vi   |
| LIST OF TABLES .....                                      | viii |
| LIST OF FIGURES .....                                     | ix   |
| LIST OF ABBREVIATIONS.....                                | xi   |
| CHAPTER 1 INTRODUCTION .....                              | 1    |
| 1.1 Background.....                                       | 3    |
| 1.2 Problem Description .....                             | 4    |
| 1.3 Objectives .....                                      | 5    |
| 1.4 Scope.....                                            | 5    |
| CHAPTER 2 LITERATURE REVIEW .....                         | 7    |
| 2.1 Historical Background .....                           | 7    |
| 2.2 Related works.....                                    | 8    |
| CHAPTER 3 RESEARCH METHODOLOGY .....                      | 21   |
| 3.1 Hardware Setup.....                                   | 22   |
| 3.1.1 Manipulator Specification.....                      | 22   |
| 3.1.2 Camera setup.....                                   | 24   |
| 3.1.3 Control Hardware.....                               | 24   |
| 3.1.4 Hardware component specifications: .....            | 25   |
| 3.2 Software Framework.....                               | 31   |
| 3.2.1 Ros Packages .....                                  | 33   |
| 3.2.2 Image Processing Algorithm.....                     | 34   |
| 3.2.3 IBVS Control Law Implementation.....                | 36   |
| 3.3 Mathematical Modeling .....                           | 38   |
| 3.3.1 Camera Projection Model .....                       | 39   |
| 3.3.2 Image Jacobian (Interaction Matrix) Derivation..... | 41   |

|                                                         |    |
|---------------------------------------------------------|----|
| 3.3.3 Depth Estimation & Uncertainty Handeling.....     | 43 |
| 3.3.4 Extensions and Practical Enhancements .....       | 44 |
| 3.3.5 Algorithm summary(per control cycle); .....       | 46 |
| CHAPTER 4 RESULTS & DISCUSSION .....                    | 49 |
| 4.1 Experimental setup Review .....                     | 49 |
| 4.2 Result of Image Processing and Object Tracking..... | 52 |
| 4.3 Depth Estimation .....                              | 56 |
| 4.4 Manipulator Control Performance .....               | 58 |
| 4.5 Comparison: Without vs. With IBVS .....             | 64 |
| 4.6 Discussion .....                                    | 65 |
| CHAPTER 5 RELATION WITH A REAL-WORLD PROBLEM.....       | 68 |
| CHAPTER 6 CONCLUSIONS AND FUTURE WORKS.....             | 70 |
| 6.1 Conclusion .....                                    | 70 |
| 6.2 Future Works: .....                                 | 71 |
| References .....                                        | 72 |

## LIST OF TABLES

|                                                                                    |    |
|------------------------------------------------------------------------------------|----|
| Table 3.1 DH parameter of the manipulator .....                                    | 24 |
| Table 4.1 Performance comparison table of used algorithms.....                     | 56 |
| Table 4.2 Comparison of the object's distance estimation to the true distance..... | 57 |
| Table 4.3 Grasping accuracy tests findings .....                                   | 63 |
| Table 4.4 Comparison with related works .....                                      | 66 |



## LIST OF FIGURES

| Figure No. | Description                                                                                                                                                                                                                                                                                                                                                            | Page No. |
|------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|
| Figure 1.1 | Spot: A Mobile manipulator capable of working in a dynamic environment.                                                                                                                                                                                                                                                                                                | 3        |
| Figure 2.1 | Classical image-based visual servoing scheme [3].                                                                                                                                                                                                                                                                                                                      | 9        |
| Figure 2.2 | A model of faster R-CNN [4]                                                                                                                                                                                                                                                                                                                                            | 11       |
| Figure 2.3 | Initial and final position of HP20D robot for path planning using Lazy PRM algorithm [5].                                                                                                                                                                                                                                                                              | 12       |
| Figure 2.4 | The autonomous industrial mobile manipulator built in Texas State University. The platform is consist with Segway RMP400 mobile platform and ABB IRB120 industrial manipulator. Gripper and different sensors are mounted on this platform [7].                                                                                                                        | 14       |
| Figure 2.5 | Motion planning of a manipulator using ROS Package in MoveIt.                                                                                                                                                                                                                                                                                                          | 19       |
| Figure 3.1 | System overview of the proposed IBVS-based manipulator control framework.                                                                                                                                                                                                                                                                                              | 22       |
| Figure 3.2 | Rendered CAD Model of the Manipulator using Solidworks 2023                                                                                                                                                                                                                                                                                                            | 23       |
| Figure 3.3 | Parameters of the physical manipulator. (a) Joint names and joint co-ordinates, (b) Dimension of links.                                                                                                                                                                                                                                                                | 23       |
| Figure 3.4 | ESP32 microcontroller                                                                                                                                                                                                                                                                                                                                                  | 26       |
| Figure 3.5 | Actuators used in the manipulator. (a) MG996 servo motor is used where high torque is required, (b) 90g servo motor is used where low torque is required.                                                                                                                                                                                                              | 27       |
| Figure 3.6 | ESP 32 CAM used for eye-in-hand vision setup of the manipulator.                                                                                                                                                                                                                                                                                                       | 29       |
| Figure 3.7 | HC-04 ultrasonic sensor used for measuring gripper-to-object distance                                                                                                                                                                                                                                                                                                  | 30       |
| Figure 3.8 | ROS node diagram of software framework                                                                                                                                                                                                                                                                                                                                 | 32       |
| Figure 3.9 | A block diagram that represents IBVS implementation on the Manipulator control.                                                                                                                                                                                                                                                                                        | 39       |
| Figure 4.1 | Physical arm test setup                                                                                                                                                                                                                                                                                                                                                | 51       |
| Figure 4.2 | Software interface for the experiment                                                                                                                                                                                                                                                                                                                                  | 51       |
| Figure 4.3 | Object tracking using CSRT algorithm and location calculating                                                                                                                                                                                                                                                                                                          | 53       |
| Figure 4.4 | Test setup for tracking a moving object using different tracking algorithms (a) physical arm setup with speed-controllable motorized object to move the object in a circular path at 30rpm speed in the manipulator's vision area, (b) camera view at the starting time of tracking, (c) camera view after 10 seconds, where KCF and MOSSE algorithms lost the object. | 54       |
| Figure 4.5 | plot of object center which is in circular motion that represent tracking performance using different tracking algorithm                                                                                                                                                                                                                                               | 55       |
| Figure 4.6 | Error and stability test of the sonar sensor measured distance for the ground truth value of 25 cm.                                                                                                                                                                                                                                                                    | 57       |
| Figure 4.7 | A setup for a grasping test. (a) Initial position, (b) Manual position setting to bring the object into the camera viewing area, (c) execution of the trajectory to move toward the object, (d) reached final grasping position, (e) software interface while executing the grasping function                                                                          | 59       |
| Figure 4.8 | Test Tracking with different types of objects                                                                                                                                                                                                                                                                                                                          | 60       |

|                                                                                |    |
|--------------------------------------------------------------------------------|----|
| Figure 4.9 Tracked object location during a closed-loop gripping control cycle | 61 |
| Figure 4.10 Trajectory of end effector                                         | 62 |
| Figure 4.11 Joint Trajectory                                                   | 63 |

## LIST OF ABBREVIATIONS

|      |                                               |
|------|-----------------------------------------------|
| PBVS | Position-Based Visual Servoing                |
| IBVS | Image-Based Visual Servoing                   |
| ROS  | Robot Operating System                        |
| MPC  | Model Predictive Control                      |
| SVM  | Support Vector Machine                        |
| CNN  | Convolutional Neural Network                  |
| PRM  | Probabilistic Road-Map                        |
| AIMM | AI Management                                 |
| URDF | Unified Robot Description File                |
| SDF  | Simulation Description File                   |
| DOF  | Degree of Freedom                             |
| CAD  | Computer-Aided Design                         |
| FLC  | Fuzzy Logic Controller                        |
| IAE  | Integrate Absolute error                      |
| GUI  | Graphical User Interface                      |
| YOLO | You Only Look Once                            |
| KUET | Khulna University of Engineering & Technology |

# **CHAPTER 1**

## **INTRODUCTION**

Since robotic manipulators are capable of tasks that are both precise and repetitive, they have become indispensable tools in numerous industries such as manufacturing, health care, the service industry, and planetary exploration. The traditional control approaches, frequently involving predefined trajectories or a priori commands, have restrictions on their flexibility. Visual servoing is the use of vision feedback the guidance of tasks executed by robots is one of the more likely methods to increase the flexibility and robustness of manipulator control systems.

Robotic manipulators operating in real-world must account for the persistent variations in lighting, object pose, occlusion, and scene dynamics. Conventional control methods require detailed system models and precise calibration, whereas fully manual control of robotic arms relies solely on the skill, reaction time, and perception of the human operator. Both are poorly suited for unstructured or rapidly changing scenarios-model-based methods lose accuracy quickly when the operating conditions do not match assumptions, while purely manual operation is time-consuming, inconsistent, and mentally challenging, especially under demands of precise positioning or real-time adjustments. These limitations emphasize the need for a hybrid control approach that combines manual selection of objects and the autonomous motion of manipulator to reach the gripping position.

Existing methods for manipulator control, which include pure model-based methods and on the other hand, fully manual control, suffer significant limitations within dynamic environments. Model-driven controllers are highly sensitive to calibration and lose accuracy within a few steps when the lighting changes, the appearance of objects changes, or some unforeseen disturbance occurs. Similarly, vision systems based on static features or along predefined trajectories have difficulty in cases where targets may move, deform, or get partially occluded. Manual operation, in turn, is burdensome for the operator and

cannot reliably produce fine, repeatable motions at high speeds. so, a more adaptive, visual and sensor feedback-driven framework is required that can adapt with any unstructured condition with minimal human intervention.

In robotic manipulation, more accuracy and flexibility in dynamic environments is becoming a critical issue. It is within this context that visual servoing (VS) emerges as a potential solution to these requirements, where VS is the process of controlling the motion of an actuator based on image-based feedback. Vision-based servoing brings higher precision and greater dexterity of actions that can be achieved when machine vision systems are combined with robotic control systems [1].

Visual servoing is based on the principle of controlling a robot in real-time using images obtained from cameras. With this visual feedback, the robot's movements are constantly corrected so as to accurately place it and follow paths. There are two common types of visual servoing that are widely followed, namely Position-Based Visual Servoing (PBVS) and Image-Based Visual Servoing (IBVS). PBVS controls the robot end-effector by an estimate of 3D position of target object, while IBVS regulates motion of robot directly based on some image features (e.g., pixel coordinates) [2].

Several benefits can be achieved with visual servoing in the manipulation context. It aids the accuracy of robot operated activities, and is suitable for any application requiring accurate positioning such as assembly, welding and surgery. Furthermore, visual servoing gives a great flexibility to the robots; a robot can quickly change its action reacting to modifications of the environment and achieve different tasks with low investment in reprogramming (Figure 1.1). Use of non-contact for visual measurements also minimizes the risk of damaging delicate or dangerous objects.



Figure 1.1 Spot: A Mobile manipulator capable of working in a dynamic environment.

The proposed work is a CSRT-based Image-Based Visual Servoing system on ROS 2 and MoveIt Servo for enhanced manipulator control. The system utilizes an eye-in-hand camera mounted on a 5-DOF robot arm and CSRT for stable tracking and ROS 2 real-time middleware for efficient perception-control communication. Experimental performance verifies improved tracking stability, reduced convergence speed, and better accuracy in dynamic environments compared to conventional IBVS implementations. This work bridges the gap between light-weight tracking and high-speed control and presents a working yet efficient solution to real-time vision-guided robotic manipulation.

The main contributions of the work are as follows:

- Development of a ROS 2-supported IBVS framework that integrates the CSRT object tracking technique for secure manipulator control.
- Facilitating a real-time visual feedback loop through MoveIt Servo to achieve smooth Cartesian motion of a 5-DOF robotic arm.
- Experimental validation with improved tracking precision and stability under dynamic environments.
- Comparative performance analysis that demonstrates the CSRT-based IBVS compared to conventional tracking approaches in aspects of accuracy and stability.

## 1.1 Background

With the rapid development of technology in modern industry, it is an essential approach to carry out lightweight and stable design on robotic manipulators to meet operation

demands with high precision and efficiency. Classification Category Classically, they rely on predefined paths and precise geometric models that work fine in orderly, predictable scenarios. Yet, these kinds of systems have to face more difficulties in a dynamic and unstructured environment where changes like sudden target movements, lighting conditions or obstacles may disturb system functionality.

With this in view, real-time adaptability formulation is an essential requirement. IBVS is one of these methods that enables the manipulator to act by using visual feedback from cameras. In contrast to position-based servoing, which is dependent on an external calibration to a great extent, IBVS operates in the image space directly; thus little dependence on models' accuracy and more independence are gained [3]. This approach has even been successfully employed in industrial robotics, medical operations, and autonomous systems.

In spite of its potential, the critical problems for IBVS are visual noise handling, computational delay and robustness to large scene changes. However, advances in machine learning and adaptive algorithms as well as computational resources improve this situation. This work extends these works for developing adaptive IBVS, with the purpose of improving the control performance of robotic manipulators in a practical sense.

## **1.2 Problem Description**

Control of a robotic manipulator, in unstructured and dynamic environments, is an extremely challenging task. Essentially, all prior approaches have relied heavily on precise calibration, predefined paths, and accurate geometric models, which in reality can never be fully adequate. In addition, changes in lighting, the addition of noise, and even unexpected target motions make the conditions difficult for precise control.

**The key issues are:**

1. **Unstructured Dynamic Environments:** The manipulators cannot cope with unstructured and dynamic surroundings.
2. **Noise and latency:** Many visual feedback systems inherently include noise and/or processing delays that degrade reliability.
3. **Stability in Complex Scenarios:** Traditional control systems get destabilized due to sudden changes in targets or obstacles.

It is in this view that the current research work is concerned with adaptive manipulator control under the concept of IBVS. Real-time visual feedback will operate to dynamically adjust manipulator movements for better precision, robustness, and adaptability. Such a system would be easily scalable for applications that require reliable robotic operations within complex environmental conditions.

### **1.3 Objectives**

The general objective of this thesis work is to develop visual servoing techniques using ROS to enhance the control of robotic manipulators.

**The specific objectives are as follows:**

- To implement efficient algorithms for real-time image processing and feature extraction within the ROS framework.
- To develop robust control strategies that maintain performance under varying environmental conditions.
- To develop a servo feedback system in all joints.

### **1.4 Scope**

Research in this area focuses on designing an adaptive IBVS framework for manipulator control in a dynamic and unstructured environment. The research scope includes:

**Framework Design:** Real-time visual feedback control strategy development for multi-degree-of-freedom manipulators.



**Adaptivity:** Deal with such issues as environmental variability, noise, and processing delays.

**Performance Evaluation:** Testing the system, both in simulated and physical environments, under different conditions, for different applications, ranging from industrial to medical robotics.

**Scalability:** The design should be flexible and adaptable to different manipulator designs and applications. The research tackles the robustness and adaptability gaps in deploying robots in real-world tasks.

## **CHAPTER 2**

### **LITERATURE REVIEW**

IBVS stands for Image-based Visual Servoing, this control method can be used to operate under dynamic and unstructured environment with the robotic manipulator. Compared with pose-based control, IBVS uses the error directly on the image plane instead of that in 3-space; therefore is relaxed to exact calibration and easier for geometric modeling. The approach itself has come a long way, with recent work on hybrid IBVS-PBVS systems that utilize the strong suits of image-based as well as position-based methods together with model-predictive control (MPC) to further enhance trajectories in dynamic situations.

Recent works have concentrated at enhancing feature detection and tracking capabilities by applying techniques like optical flow and machine learning to have robust and adaptive operation. IBVS has been applied not only to industrial automation (assembly and pick-and-place applications), but also in specialized areas such as medical robotics for minimally invasive surgery and autonomous systems for UAVs or underwater vehicles. But it is distorted by noise, occlusion, latency and inaccuracy of depth estimation so that the reliability and precision decrease in dynamic scenes.

Towards these conquests, explorations like the scalable model that combines reinforcement learning and predictive filtering for noise suppression to enable real-time adaptivity has been made. There still remains a wide research gap in the development of scalable, general-purpose IBVS systems that retain their performance when being applied to unstructured environments such as disaster response or outdoor exploration. Hence, to overcome these limitations the work proposed in this paper will fill them by presenting an adaptive IBVS scheme to enhance control of robotic manipulators in real-world complex tasks making it possible for robots to navigate through dynamic environments.

#### **2.1 Historical Background**

Visual servoing field was born at the end of last century, which offered a solution for accurate and flexible robot manipulators by merging the vision system with motion control. The early start on visual servoing was made in the 1980's, with prominent researchers as

Sanderson and Weiss making leading-edge contributions to the camera feedback control of robot arms.

For instance, Image-Based Visual Servoing developed in the 90s really leapfrogged the state of control for manipulators[4]. The work in IBVS aimed to control the motion of robotic arms using the error signals defined on the image plane without requiring an explicit 3D model. This new approach is much more flexible and real-time than position-based methods.

As both computational power and vision technologies got better, researchers began addressing these problems and more: namely noise, latency and difficulty of depth estimation (particularly for dynamic environments). 2 Again, such problems have been addressed to by the introduction of hybrid approaches in which IBVS was integrated with PBVS. More recently, the application of machine learning methods, especially reinforcement learning (RL) and feature tracking with deep learning, has further increased the robustness and adaptability of IBVS leading to expanded use beyond academia such as in industry automation, medical robotics and self-driving cars.

The progress of IBVS is an instance traceable back to a general trend: the production of increasingly flexible, autonomous equipment able to work in complex and unstructured environments. Continuing along this historical perspective, with this thesis we aim at furthering manipulator control through adaptive IBVS methodologies specifically tailored for real-time dynamic applications.

## **2.2 Related works**

Visual servoing can be broadly classified into two categories: image-based visual servoing (IBVS) and position-based visual servoing (PBVS). IBVS uses image features for control structure that is not the case with PBVS.

Garcia, Pomares, Gil et al. (2013) introduced “Event-Based Visual Servoing: Tracking the Features’ Prediction” shown in Figure 2.1 [3]. They propose a computational model of

detecting and localizing salient objects in images. The model is influenced by the human visual system's visual attention mechanisms. It includes a saliency map generator to generate the saliency map from an input image, and an object detection module for searching the most salient regions in the generated saliency map.

The generator of saliency map: It adopts a multi-scale feature extraction manner, and obtains the low-level features of color, intensity and orientation at different spatial scales. These feature maps are aggregated to form a final saliency map, emphasizing the most prominent regions in the image.

A sliding window approach is adopted by the object detection module to find meaningful candidates within the saliency map. It is based on training a linear SVM classifier to label the sliding windows with salient or not. The classifier is learned on a dataset of labeled salient objects.

The proposed model is tested on a number of benchmark datasets, and its performance is compared with other existing saliency detection and object localization methods. Experimental results demonstrate that the proposed model can achieve competitive performance in both accuracy and computational efficiency.

The paper also addresses some possible applications of the proposed model to image segmentation, content aware image editing and visual search.

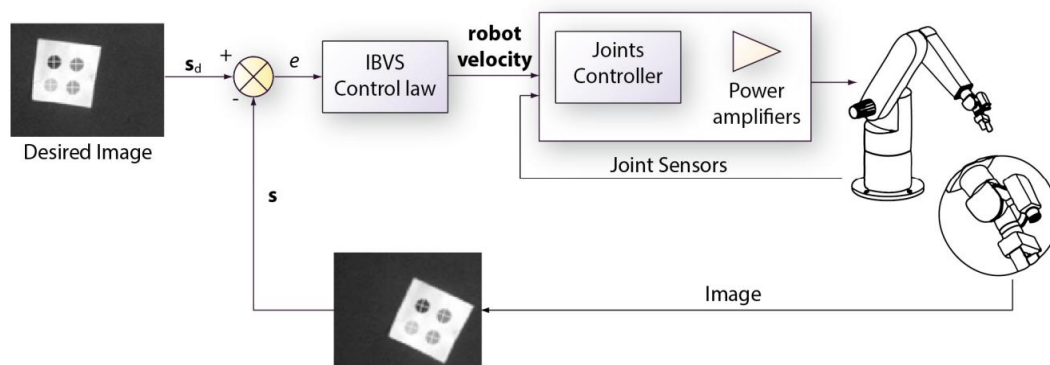


Figure 2.1 Classical image-based visual servoing scheme [3].

More recent studies have investigated how advanced image processing techniques and deep learning can be used to increase the precision and speed of feature recognition and object tracking. Nevertheless, such methods generally face issues of high computer cost or environmental changes for real-world scenario use.

Chen, Guhl et al. (2018), “Industrial Robot Control with Object Recognition by Deep Learning.”[4]. They give a template for the coordination of deep learning based object recognition with industrial robot control. The aim is to provide robotic systems with the capabilities needed for more flexible and adaptive robot systems that can handle a variety of objects, as well as different tasks.

The proposed architecture has two main parts:

Object recognition: We implement an object recognition module on a deep CNN model for detecting and classifying objects in the robot's environment. The CNN is learned from a set of object images to identify various types of objects (Figure 2.2).

Robot control module: The output of the object recognition module is input to this module, which issues the correct set of control commands for the robot. This provides the core top-level commands and APIs used to implement object pose estimation, path planning, and robot motion control.

The setup of these two modules is described in the paper. Regarding the object recognition, they try different CNN structures and training methods to get a better accuracy of recognition. For the robot control, they propose methods to estimate the 6D pose of all detected objects and generate collision-free robot trajectories.

Evaluation of the framework is based on a real industrial robot system. It demonstrates that high precision deep learning based object recognition can be achieved, and the robotic manipulator in the integrated system can also successfully pick and place a variety of objects.

Benefits of the proposed method in comparison with conventional robot programming (e.g., higher flexibility, less tedious programming and capability to cope with more complex and unstructured environments) are presented by the authors.

In conclusion, this research highlights how deep learning techniques can extend the reach and skill set of industrial robots, leading to more flexible intelligent robotic systems.

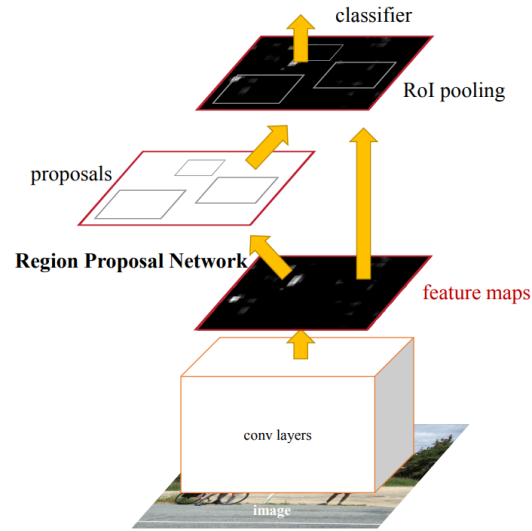


Figure 2.2 A model of faster R-CNN [4]

Path planning and visual servoing for robotics is important. The PRM was originally proposed by Kavraki et al. (1996), and its modifications such as Lazy PRM exhibits strong performance in high-dimensional space for collision-free motion. Additions, including dynamic node insertion and collision resolution further improve efficiency (Bohlin & Kavraki, 2000) [5]. By contrast, visual servoing – in particular image-based methods such as IBVS (Espiau et al., 1992) – guarantees accurate control of the end-effector based on the most recent local view (Figure 2.3).

Although PRM has been applied to many problems such as molecular modelling and robot navigation (Al-Bluwi et al., 2012), the combination with visual servoing extends its versatility in manipulation. But, it suffers from a computational overhead of real-time path corrections and solid homography estimation for correct visual feedback.

The fusion of PRM and IBVS bridges a gap in single approaches of either global obstacle avoidance or local positionability. The testing using ROS makes the approach feasible for real warehouses and dynamic industrial environment [5].

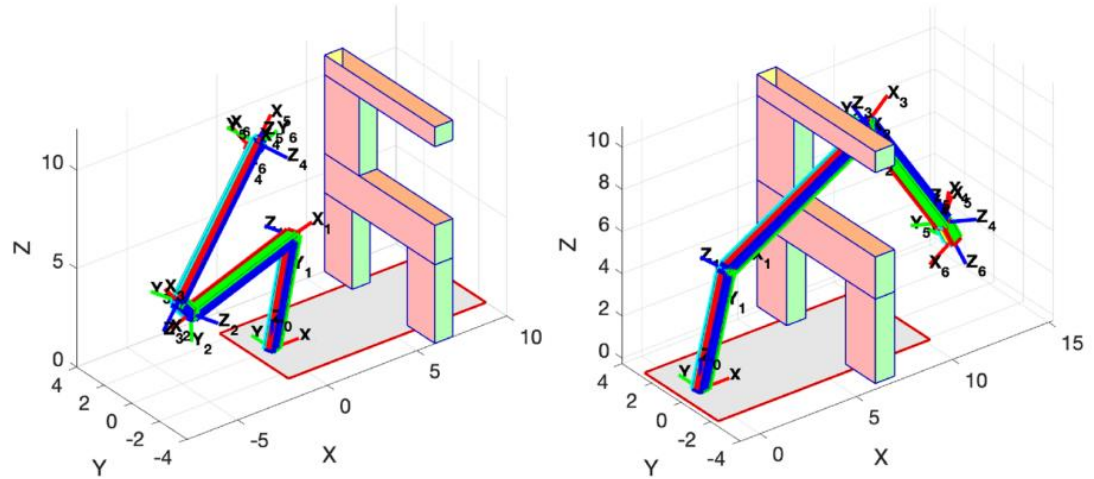


Figure 2.3 Initial and final position of HP20D robot for path planning using Lazy PRM algorithm [5].

The paper Manipulator Control With Image-Based Visual Servo, Hashimoto et al. (1991) proposed the idea of image based visual servo (IBVS), where robotic manipulator movements are controlled directly using image features. This was a departure from classical position-based servoing, which relied more heavily on explicit position and orientation calculations and consequently improved system robustness.

In addition, some attempts of them have been made for position-based visual servoing based on precise object localization and depth estimation (Weiss et al., 1987). But these approaches are not robust under noisy sensors, and weak in the unstructured environments. Methods, such as IBVS, which are image-based could attempt to alleviate some of these issues by trying to directly minimize errors on the image plane (and have been successful under dynamic tracking and industrial settings).

Although IBVS presents a computationally efficient approach compared to position-based techniques, the fact that real-time image feature extraction and Jacobian matrix calculation

is still required indicates that there are computational trade-offs between IBVS and system response time. Recent work has concentrated on the hybrid visual servo methods that blend the advantages of image-based and position-based control methodologies in coping with complex and dynamic environments [6]. Combining the PRM and IBVS fills a need over independent approaches where global reactive obstacle is separated from local positional accuracy. Experimental Validation on ROS By experiment validation using the Robot Operating System (ROS) it makes the method practical and feasible to apply for in-warehouse work and where there is dynamics industrial environment [5].

Autonomous industrial mobile manipulators (AIMMs) are revolutionizing the mobility of mobile robots with the accuracy of industrial manipulators, as well as contributing to applications in dynamic and hazardous environments. Typical stationary robots are ineffective in unstructured spaces, and there is a new challenge: improving the coordination, sensing, and calibration of current systems.

The work of Cheng, Chen and Liu (2013) tackles these limitations, considering object manipulation with AIMMs with RGB-D cameras for depth estimation and world understanding. It reveals the limitations of the available IBVS approaches with a monocular camera and introduces a PBVS technique to improve handling and manipulation capabilities (Figure 2.4).

Although this study reveals remarkable progress in AIMM control and coordination, combined advanced algorithms (for simultaneous navigation and manipulation) could be



integrated to improve the efficiency. Moreover, dynamic problems such as real-time obstacle avoidance and multi-object manipulation are not well-addressed [7].

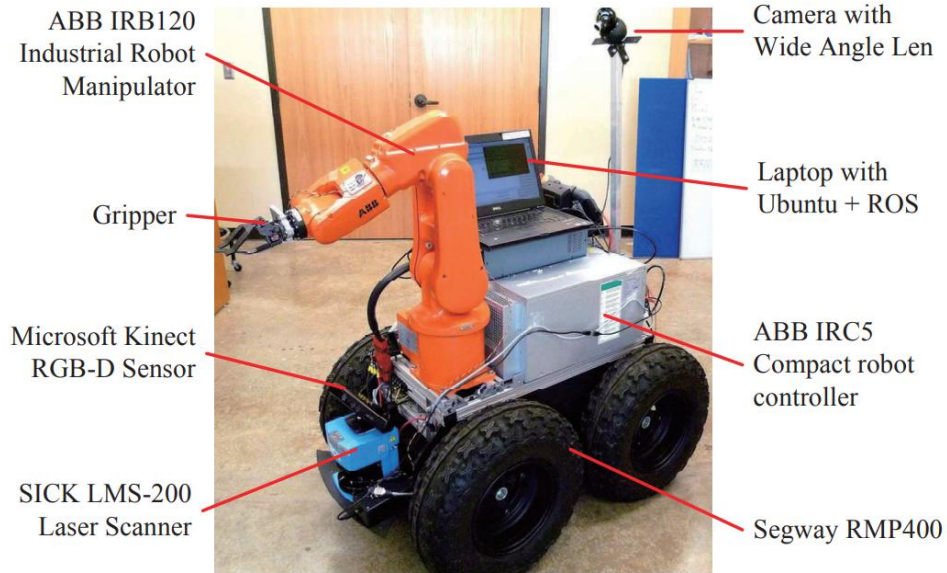


Figure 2.4 The autonomous industrial mobile manipulator built in Texas State University. The platform is consist with Segway RMP400 mobile platform and ABB IRB120 industrial manipulator. Gripper and different sensors are mounted on this platform [7].

ROS 2 configuration for Delta robot arm kinematic motion and stereo camera visualization Saipullah et al. (2022) describes the design and simulation of a delta robot with the ROS 2 framework. Delta robots, which are high speed and precision robots, have been used extensively in industrial and agricultural operations. In this study, we present an end-to-end simulation workflow for Delta robot arm that includes kinematic motion with stereo camera rendering in ROS 2.

The work demonstrates the limitations of URDF (Unified Robot Description Format) to model the closed-loop kinematic chain of the Delta robot, so that SDF (Simulation Description Format) was used instead by the authors. By Gazebo simulator, the robot's physical properties were well simulated and studied to simulate a realistic environment forces and interactions.

One of the novel concept underlying this work is how to reconstruct image depth and stereo vision was used by using a virtual stereo camera connected to Delta end-effector. The incorporation of stereo visualization through Rviz 2 demonstrates the potential for improved depth perception which has applications in robotic tasks requiring accuracy, e.g. object grasping.

Moreover, the inverse kinematic motion technique has been utilized for accurate positioning of joint with error percentage herein averaged 3.92%, 3.72% and 2.92% for first three joints respectively. The research also promoted the modular features of ROS 2, as they were made using its plugins library to simplify robotics movement and visualization.

The value of this work lies in the development of prototypes that bring theory to practice, as relevant to simulation-based robotics. Regarding relevance to the field and novelty, the authors state that these methods are generalizable by developing custom Gazebo plugins and incorporating more advanced image processing techniques (e.g., OpenCV) for training data generation [8].

The paper Design and Implementation of Robotic Arm with ROS and MoveIt! by Hernández-Méndez et al. (2017) introduces a 3-DOF robot arm and its control integrated in ROS (Robot Operating System) to be used with MoveIt! for object manipulation. The latter contribution demonstrates the significance of motion planning and hybrid position/force/mp control for tasks incontrolinction with the robotic arm. Authors developed robotic arm with the use of FreeCAD, created URDF files for visualization and integration into ROS.

Critical modules are the Dynamixel actuation elements to control each joint, piezo-electric sensors for force feedback, and hybrid PID controller to fulfill safe and non-damage object manipulation. The robot was also tested in a semi-structured environment where the utility of the motion planning algorithm RRT-Connect in OMPL for successful trajectory tasking and execution were demonstrated. In the experiments, the grasping motion, pick-and-place and release of an object were confirmed to be performed with high accuracy utilizing the hybrid controller.

The designing of hardware and software using ROS and MoveIt! demonstrates the modularity and flexibility of our tools in robotics. The findings in this paper pave the way toward integrating more functionality such as computer vision for object detection and localization in future work [9].

Yau et al. (2014) studies advanced control techniques to improve the transient positioning performance of a DC servo motor. The work combines Evolutionary Programming (EP) and Genetic Algorithms (GA) optimized PID controllers, later tuned by a Fuzzy Logic Controller (FLC). This mixed method seeks to overcome the limitation of classic PID controllers in motor optimization.

The authors suggest a two-step optimization process. In the first stage, EP and GA are used for setting the PID gains by means of the IAE cost function. Then, FLC with fuzzy rule sets and membership functions is utilized to adjust the gains for precise control. Experimental validation, employing a Quanser SRV02 DC servo motor confirms that the FLC ameliorated PID controllers substantially enhance settling time. Particularly, convergence times are reduced and better transient response is achieved when the FLC+EP based controller over the FLC+GA method.

This low level controller represents the ability of coupling evolutionary algorithms and fuzzy logic in order to achieve high accuracy on modern motor control and demonstrates its effectiveness in scenarios where robustness and responsiveness are necessary. The work concludes by comparing the advantages of EP and GA in optimizing dynamic systems and the impact of FLC to improve performance [10].

The study by Huang et al. (2020) presents a comprehensive framework for simulating and modeling a six degrees of freedom (6 DOF) robotic arm using the Gazebo simulation environment. The paper emphasizes the importance of simulation systems in verifying planning algorithms and analyzing the dynamic characteristics of robotic manipulators in various operational scenarios. Gazebo's integration with the Robot Operating System (ROS) is leveraged to enhance the system's flexibility and adaptability, particularly in handling different robotic models.

The paper proposes a structured system architecture comprising three key components: model configuration, data processing, and graphical user interface (GUI). This modular

approach simplifies the simulation process and facilitates the customization of the robotic arm model. The use of the Unified Robot Description Format (URDF) and other configuration files ensures precise definition of physical attributes and kinematic constraints. The integration of Gazebo's physical engine enables accurate simulation of collision dynamics and motion characteristics, addressing limitations noted in alternative platforms like MATLAB and RVIZ.

A significant highlight of the study is the implementation of an enhanced Rapidly-exploring Random Tree Star (RRT\*) algorithm for path planning. By introducing target-oriented features and avoiding abrupt angular changes, the algorithm optimizes the smoothness and efficiency of planned paths. Additionally, the authors use the Parallel-Axis Theorem to accurately estimate moments of inertia (MOI), a critical parameter for realistic physical simulations [11].

The paper by Megalingam et al. (2020) addresses the integration of vision-based manipulation with robotic arms using the Robot Operating System (ROS) for assistive applications. The research proposes a system designed to assist individuals with disabilities or the elderly in daily tasks. The integration of YOLO v3, a convolutional neural network-based object detection algorithm, with a Kinova Jaco robotic arm forms the core of the system.

The paper provides a detailed framework for the seamless integration of object recognition and robotic manipulation within the ROS environment. YOLO v3 enables real-time recognition and localization of objects, which are then translated into manipulator commands through a transformation of reference coordinates between the Kinect camera and the robotic arm. This transformation ensures precise positioning and motion control of the arm, leveraging inverse kinematics and ROS-based tools like MoveIt for motion planning.

This research addresses challenges in assistive robotics by improving autonomy and usability. The use of YOLO v3 for robust object recognition and the integration of multiple control frameworks demonstrate an innovative approach to enhancing robotic functionality. The proposed system emphasizes the practical implications for assistive robots, such as service robots, wheelchairs, and rescue systems [12].

The study by Huang et al. (2020) developed an overall simulation and modelling of six degrees of freedom (6 DOF) manipulator robot with the gazebo environment. The work highlights the role of simulation systems for the validation of planning algorithms and the study of the dynamic behaviour of robotic manipulators in different working conditions. Gazebo is interfaced with the Robot Operating System (ROS) to improve system flexibility and versatility, especially for deployment on a variety of robot models.

A structured system architecture is presented that consists of the model configuration, the data processing and the graphical user interface (GUI). Such a modular representation offers the flexibility in simulating and customizing robotic arm model. URDF and other configuration files are utilized to accurately define the physical characteristics and kinematic properties. Incorporation of Gazebo physical engine allows accurate simulation in both collision dynamics and motion properties, correcting deficiency found in other platforms such as MATLAB and RVIZ.

As a key feature of the study, an improved RRT\* (Rapidly-exploring Random Tree Star) algorithm is applied for path planning. By adding target-driven aspects and smoothing stepwise-turn, the algorithm ensures smoothness and efficiency of planned trajectory. And the Parallel-Axis Theorem is also adopted by authors to accurately predict MOI, an important factor for realistic physics simulation [11].

The article of Megalingam et al. (2020b) on the integration of vision-based manipulation along with robotic arms using the Robot Operating System (ROS) for assistive purposes. The work suggests a system that helps disabled/elderly to perform daily routine activities. A combination of YOLO v3, which is a CNN-based algorithm to detect an object with Kinova Jaco robotic arm for the actuator was used to develop this system.

The paper also presents a structured methodology to easily plug in both object recognition and robotic manipulation capabilities into the ROS system. With the YOLO v3 detection method, we are able to perform simultaneous detection and localization in real time and process them to be manipulator commands using the transformation of reference coordinates from both Kinect camera and robotic arm. This conversion provides the

accurate placement and orientation of the arm and the corresponding motion, using inverse kinematics as well as ROS-based tools such as MoveIt for path generation (Figure 2.5).

This work mitigates the limitations for assistive robots in terms of autonomy and user experience. Application of YOLO v3 to robust object recognition and combination with multiple control frameworks, indicates novelty in improvement of the functionality of robots. The prospective system focuses on practical applications as assistive robots, e.g., service robotics, a wheelchair and a rescue systems [12].

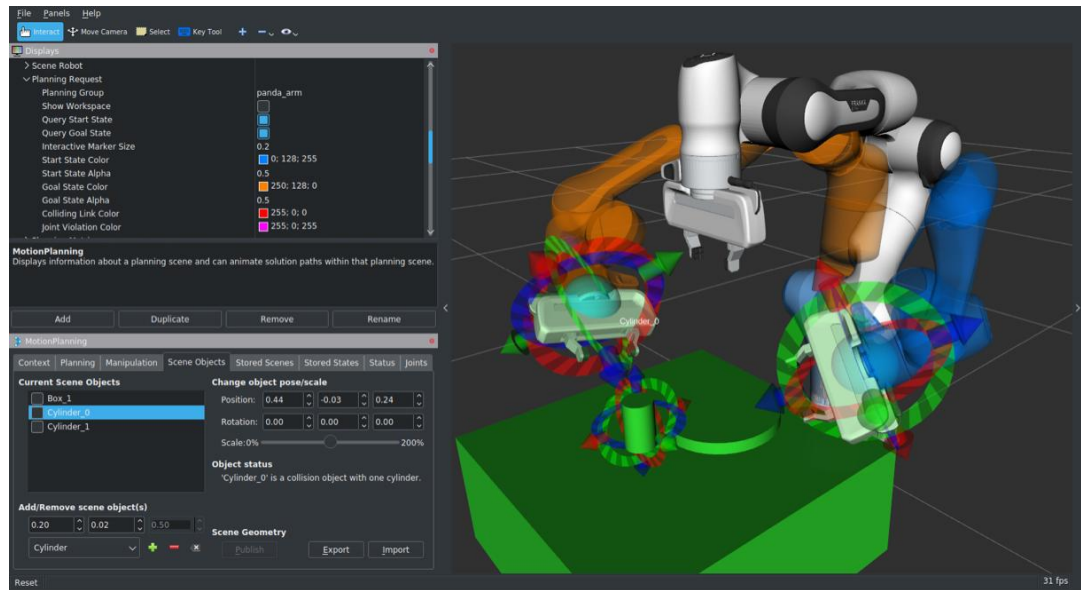


Figure 2.5 Motion planning of a manipulator using ROS Package in MoveIt.

The theoretic origins of visual servoing date back to Hashimoto et al. (1991), who first mathematized image-based visual servo control (IBVS) with an eye-in-hand camera configuration [13]. This was built upon by Corke's research that merged control theory, vision, and robot dynamics into a single visual servoing [2], and Malis et al., who introduced the hybrid 2D/3D approach that improved convergence and reduced the utilization of full 3D models [14]. Marchand et al. generalized IBVS to be used with manipulators without proprioceptive sensors, using visual feedback only for the control of underwater robots [15]. Corke and Hutchinson then proposed a partitioned IBVS solution that decoupled translation and rotation to achieve smoother paths and improved feature

preservation in the image plane [16]. Subsequent advancements are Tsai et al. utilizing light field cameras to augment IBVS with dense spatial and depth details, providing occlusion resistance and limited field of view [17].

Alongside such control improvements, object tracking came of age as a critical component in IBVS pipelines. Bolme et al. presented the high-speed MOSSE adaptive correlation filter for robust tracking against scale and appearance changes [18]. Henriques et al. took this further with Kernelized Correlation Filters (KCF), applying Fourier-domain computation for online training at high speed [19]. Danelljan et al. followed with the development of the ECO tracker, which maintained state-of-the-art accuracy without reduced model complexity [20]. Tang et al. then improved computational efficiency further by eliminating boundary effects in correlation filters with their nBEKCF model [21]. Bertinetto et al. later introduced Fully-Convolutional Siamese Networks for offline feature embedding and real-time tracking without online update [22].

These works have been complemented by domain-specific applications. Shamshiri et al. presented visual servoing for farming robots, which highlighted the small RGB-D sensors and open-source vision software in enabling autonomous pruning, harvesting, and spraying [23]. Similarly, recent exhaustive publications [24], [25], [26] have identified new robotic perception and control techniques applied in different industrial and collaboration environments. Visual servoing and deep learning keep advancing—Deep Visual Servoing Feature Network (DVSFN) ensures reliable grasping with scale-invariant features and sparse attention for occlusion situations [27]. Likewise, similar homography-based IBVS has found successful application in similar cost-effective automation on mobile object delivery robots [9] and vision-based grasping systems have been proven effective for adaptation to unknown objects with no full 3D models [28]. The DCSFC-Grasp network [29] also contributes to grasp detection using deformable convolution and spatial feature centers for effective multi-object manipulation in dynamic settings.

## CHAPTER 3

### RESEARCH METHODOLOGY

The approach to this study is to develop hardware design, vision-based perception, and control algorithms towards a single Image-Based Visual Servoing (IBVS) framework as enabled through ROS2 (Figure 3.1). A 5-DOF computational model of a robotic manipulator is created with the help of servo motors driving their equivalent D-H linkages and an onboard eye-camera system providing visual feedback. The camera video stream is directly hitting the workstation running ROS2 and onboard OpenCV-based image processing to detect and track the target object. The centroid position of the sensed object is the main image feature for control. The IBVS control law is then used to control the camera velocity according to the error in feature position from its desired location using the image Jacobian. The velocity command is then transformed to joint velocity commands via the manipulator Jacobian, and executed with MoveIt Servo in ROS2 [30]. The computed joint commands are then sent to the ESP32 microcontroller over a wireless link, which produces PWM signals for the servo motors of the manipulator. The loop runs in a continuously manner making the end-effector to converge towards the target in image plane corresponding to a closed-loop visual control. The approach does not only allow real-time operation of IBVS but also can provide you with the possibility to compare between eye in hand and eye to hand implementations validating by that the robustness that control scheme may inherit when coping with different operating conditions.

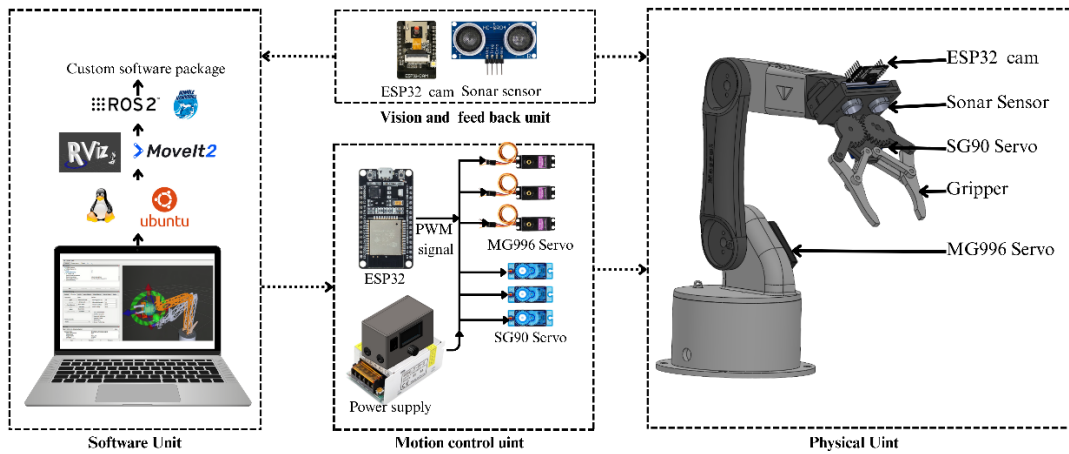




Figure 3.1 System overview of the proposed IBVS-based manipulator control framework.

### **3.1 Hardware Setup**

The experimental setup of this research consists of three primary elements: the manipulator, the vision system, and the control hardware. These elements form a closed-loop system to enable real-time Image-Based Visual Servoing (IBVS) of a robotic arm in the ROS2 environment [31].

#### **3.1.1 Manipulator Specification**

The manipulator used in this work is modeled in SolidWorks (Figure 3.2) and a custom-made 3d printed five degrees-of-freedom (5-DOF) robotic arm that would be agile enough for vision-guided manipulation but, on the other hand, light in weight and inexpensive. The arm features a rotating base, a shoulder joint, an elbow joint, a wrist joint, and a gripper as an end-effector (Figure 3.3), and the physical parameters of Table 3.1 are used in software work. Actuation is provided with a combination of servo motors selected depending on the torque requirement of each joint. High-torque MG996R servo motors are used in the base, shoulder, and elbow joints to enable the lifting and extension of the arm, while miniaturized SG90 servo motors are used for the wrist joint and gripper due to their smaller size and lesser torque requirement. For providing an added sensory feedback while operating, a low-cost ultrasonic sonar sensor (HC-SR04) is installed near the gripper. The sensor offers the manipulated object approximate depth knowledge, which is particularly useful in depth uncertainty compensation for IBVS control. The chosen actuators and sensors coupled allow the robotic arm to execute precise visual servoing tasks such as tracking and aligning objects in real-time.



Figure 3.2 Rendered CAD Model of the Manipulator using Solidworks 2023

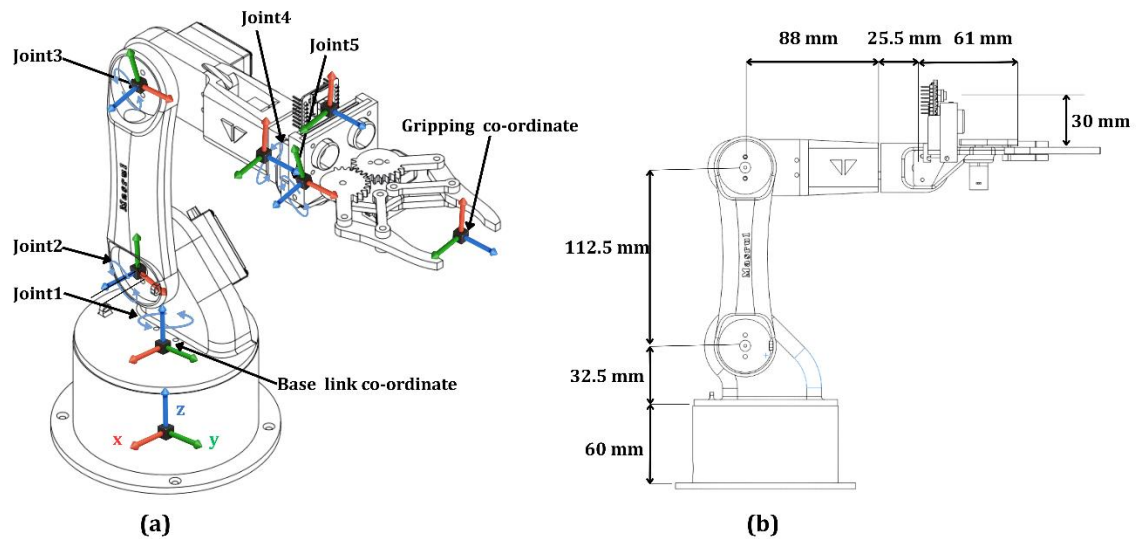


Figure 3.3 Parameters of the physical manipulator. (a) Joint names and joint co-ordinates, (b) Dimension of links.

Table 3.1 DH parameter of the manipulator

| Link | Twist              | Length     | Offset     | Joint Angle        |
|------|--------------------|------------|------------|--------------------|
| i    | $\alpha_i$ (dgree) | $a_i$ (mm) | $d_i$ (mm) | $\Theta_i$ (dgree) |
| 1    | $-\pi/2$           | 0          | 92.5       | $\Theta_1$         |
| 2    | 0                  | 112.5      | 0          | $\Theta_2$         |
| 3    | 0                  | 88         | 0          | $\Theta_3$         |
| 4    | $\pi/2$            | 0          | 25.5       | $\Theta_4$         |
| 5    | 0                  | 0          | 61         | $\Theta_5$         |

### 3.1.2 Camera setup

A camera system is utilized to provide visual feedback that is central to IBVS. The principal vision sensor is an ESP32-CAM module, chosen due to its compact size and Wi-Fi streaming capability. The ESP32-CAM delivers a live video output at a resolution of  $640 \times 480$  pixels with a camera frame rate of around 20–25 frames per second, offering a compromise between computational requirements and image clarity. For most experiments, the camera is mounted on the manipulator's wrist to create an "eye-in-hand" configuration with the end-effector having a direct view of the target object. This configuration is particularly well-adapted to fine alignment operations, as the camera image follows in real time the motion of the manipulator. Besides ESP32-CAM, a single external IP camera was also utilized in some experiments to provide an "eye-to-hand" setup, where the camera is fixedly mounted in the environment and observes the entire workspace. With two setups, comparative evaluation of both eye-in-hand and eye-to-hand IBVS methods was performed. Object detection and tracking were performed using image processing methods applied on the input camera, where the object center was taken as the dominant feature for visual servoing.

### 3.1.3 Control Hardware

System control is distributed between a host PC and a microcontroller. At low level, the system utilizes an ESP32 microcontroller for servo motor control through PWM signals. ESP32 is also a wireless communication interface that receives joint angle commands from the host computer via Wi-Fi. There is a light-weight web server implemented on the ESP32

to receive and execute control commands in real time with zero latency. At the high level, vision processing, computation, and control law execution are all performed on a Linux workstation running ROS2 Humble. The workstation is the core node of the system, which consists of perception and motion planning modules. It specifically runs MoveIt Servo as a real-time manipulator controller, the image\_transport package to deal with camera feeds, and special control nodes for IBVS implementation. Communication between the camera, host machine, and microcontroller is achieved through a wireless network, wherein video data is transmitted from the ESP32-CAM to the workstation and joint commands are transmitted in return from the workstation to the ESP32. This setup achieved an effective control rate of approximately 10–15 Hz, which was sufficient for seamless visual servoing of the 5-DOF manipulator.

### **3.1.4 Hardware component specifications:**

#### **Microcontroller (ESP32):**

The hardware architecture of the proposed IBVS system hinges on the ESP32 microcontroller (Figure 3.4), which functions as the low-level controller for the robotic manipulator. The ESP32 is a highly versatile, low-cost system-on-chip designed and manufactured by Espressif Systems and features a dual-core Tensilica LX6 microprocessor with clock speeds of up to 240 MHz, which is ideal for real-time embedded applications. Most importantly, the ESP32 features integrated Wi-Fi and Bluetooth connectivity, which negates the requirement for additional communication hardware and enables wireless data transfer with a host computer. In this research work, the ESP32 is used primarily to receive the joint angle or joint velocity commands wirelessly from the ROS2 workstation and convert such commands into PWM signals to drive the servos in the manipulator. The ESP32 integrates a large number of general-purpose input-output pins and dedicated PWM channels that make it particularly suitable to control a multi-DOF robotic arm with multiple heterogeneous servos. Additionally, the ESP32 functions as the control end for the ultrasonic modules and other sensors to provide feedback. The ESP32 also supports a lightweight web server, and in this research work, it has been used for the communication interface between the ROS2 framework and the manipulator, which provides a distributed control architecture with minimal latency. The ESP32 is a perfect solution for closed-loop vision-directed control of robotic manipulators due to its low cost, small form factor, low

power requirement, and versatile communication option and is well suited for embedded robotics research.

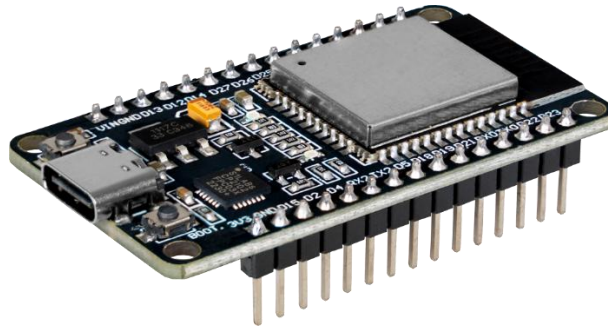


Figure 3.4 ESP32 microcontroller

### **Servo Motors**

Servo motors are the basic actuation blocks of the manipulator, they permit precise control over the five degrees of movement needed for image-based visual servoing applications. A servo is a small, sturdily-built electromechanical device with a DC motor (and sometimes gear reduction) fitted with a feedback potentiometer and control circuit all within the same housing. Servo motors are designed for more precise angle control than is required by normal dc motor, and so they find use in robotic manipulators. The motor is controlled via Pulse Width Modulation (PWM), and the duty cycle of the input signal dictates the output shaft position. Most hobby standard servo motors have a control signal frequency of 50 Hz and the duty cycle is between 1ms to 2 ms which represents rotation angle from around  $0^\circ$  to approximately  $18^\circ$ .

In this work, MG996R and SG90 servo motors are used in combination as their mix brings a trade-off among the necessary torque, small size and cheap cost (Figure 3.5). The MG996R high-torque digital servo/actuator is a popular choice for small to medium size robots and for its price, the MG996R is hard to beat! With metal power gears which make it great for applications like the humanoid part, the manipulator car etc. An MI ArTIC response time and holding torque are fast enough for stable operations during dynamic

motion which is key for smooth IBVS performance. The SG90, in turn, is a low torque micro servo motor with a stall torque of  $\sim 1.8 \text{ kg}\cdot\text{cm}$  at 5 V, hence it is less powerful than the MG996R, but given its small size and reduced current usage can be appropriate for joints that do not require much load such as wrist or gripper.

The manipulator is strong and agile with ease of access due to its modular low-cost design using MG996R and SG90 motors. The MG996R is for the stability and are in(c)ity on heavy primary joints, while we use the SG90 to have control for very small fine movements of the end-effector. These drive the five degrees of freedom of the manipulator in good coordination with each other, allowing directable reaching, orienting and grasping on visual feedback. Moreover, the PWM signals to control servos are produced from ESP32 microcontroller in order to transmit real-time servo controls of a ROS2 framework.

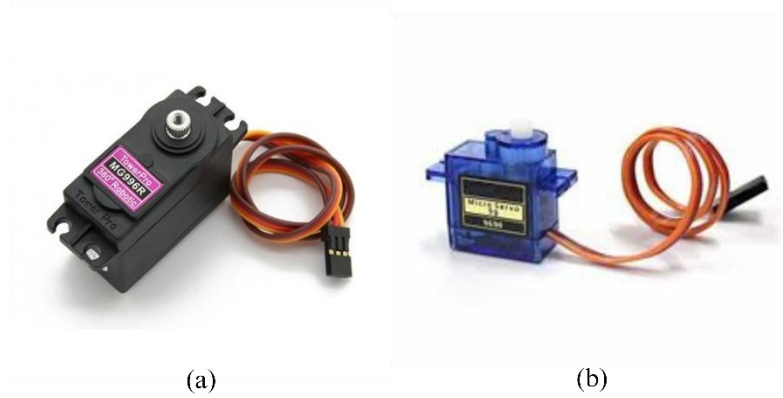


Figure 3.5 Actuators used in the manipulator. (a) MG996 servo motor is used where high torque is required, (b) 90g servo motor is used where low torque is required. required

### ESP32-CAM

ESP32-CAM (Figure 3.6) is used as the main visual sensor of the robot arm controller system which supplies real-time image feedback required by Image-Based Visual Servoing (IBVS). “This is an ultra-low power consumption development board for a WROVER module, produced by the Chinese company Espressif,” Bugezm writes on the project page. The ESP32-S is a wireless module based on ESP32, supports WiFi and Bluetooth 4.2, with two high-performance 32-bit LX6 CPU cores (based on Tensilica Xtensa LX6 core), performing at up to 600 DMIPS. 2 BR/EDR and BLE) communications. These

characteristics make the ESP32-CAM an ideal choice for wireless image transmission in robots or other applications where the size and distance are important factors. The resolution of the onboard camera can be set between  $160 \times 120$  and  $1600 \times 1200$  pixels, with a maximal streaming frame rate between 20–25 frames per second at intermediate resolutions (e.g.,  $640 \times 480$ ), as a compromise between image quality and computational burden for real-time visual tasks.

For our solution, the ESP32-CAM is installed at the wrist of the manipulator forming an eye-in-hand configuration which keeps its field of view following the movement of the end-effector dynamically. This position enables the manipulator to keep the object in its view throughout tracking and grasping operations. Video is transmitted through ROS2 to the workstation computer and frames are grabbed using the `image_transport` package for processing with OpenCV based algorithms for feature detection and tracking. The centroid of the tracked object in the image plane is taken as main visual feature and then given to IBVS control law. Through real-time images feature and closed-loop feedback, the ESP32-CAM is instrumental in directing the manipulator to its goal.

The ESP32-CAM even features a web server you can run on the microcontroller itself ditching the need for any (more) additional transmission components. In this system, the module transmits video via wi-fi directly to the ROS2 world where it can be subscribed to as a ROS2 image topic. The low-priced and compact ESP32-CAM has already demonstrated potential for performing visual servoing in an indoor setting. Its disadvantages include small field of view and no depth sensing ability by default, which required us to add our own ultrasonic sensor as an approximate range feedback source in this work.

In conclusion, the ESP32-CAM offers a cheap and low-consumption embedded solution for real-time vision processing in robotic manipulators. Through wireless ROS2, it's made modular and flexible so that the system can be utilized in eye-in-hand as well as eye-to-hand. In the proposed system, the ESP32-CAM has all of sensory information necessary to

do closed-loop IBVS and can yield accurate manipulation with adaptive law based on visual feedback.

In short, the hardware architecture is composed of a five degrees-of-freedom servo motor driven robotic arm, vision system with ESP32-CAM and IP cameras and distributed controller that uses both an ESP32 microcontroller and ROS2 based workstation. The modularity of the hardware allows for experiments to be performed under eye-to-hand and eye-in-hand configurations with a cost effective, small-scale system capable of real-time IBVS experimentation.

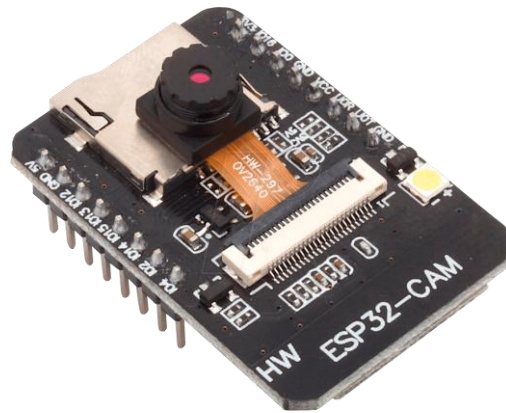


Figure 3.6 ESP 32 CAM used for eye-in-hand vision setup of the manipulator.

### **Sonar Sensor**

To partly address the limitations of vision and exploit its main pros, and to outperform the drawback of monocular camera in depth estimation, an ultrasonic sonar sensor (HC-SR04) is embedded into the designed manipulator control system (Figure 3.7). HC-SR04 is a widely used ultrasonic ranging module whose range is from 2 cm to 400 cm, the accuracy of measurement can be up to  $\pm 3$  mm. It is based on the principle of ultrasonic echo ranging, which transmits an ultrasonic pulse at 40 khz from its transmitter and measures the time it takes for the echo to return to the receiver when hitting an object. This time-offlight measurement is subsequently converted to distance by the known propagation velocity of



sound in air. It's a simple piece of hardware which enables HC-SR04 to deliver real-time distant measurements.

In this study, the sonar sensor is integrated close to the gripper of the manipulator and detects distance from end-effector to destination object programmatically. This placement also naturally corresponds to the depth information in the manipulator's working area, which is extremely relevant for grasping and approach. The sensor generates digital signals of trigger and echo pulses that are read by the ESP32 microcontroller to obtain the distance from target. This information is sent back to the ROS2 Workstation where it can be fused with visual feedback from ESP32-CAM. The inclusion of the sonar data adds another perception sense which can be valuable in IBVS problems where monocular vision is unable to accurately get scale or depth.

The sonar sensor in the feedback loop is used to improve robustness to position uncertainty of the visual servoing system. For instance, when the arm approaches an object, those depth readings from the HC-SR04 can control overshooting or avoid hitting something you like, in that way safer and more reliable to keep things going. While ultrasonic sensors are constrained by issues like sensitivity to soft or irregular objects and poor accuracy in the presence of noise, their low-cost, low power nature - as well as adequate precision for a small scale robotic platform - make them an ideal choice for this project.



Figure 3.7 HC-04 ultrasonic sensor used for measuring gripper-to-object distance

### 3.2 Software Framework

The developed manipulator control system is implemented using the layered software model to facilitate integration of robotics middleware, vision libraries, control algorithms and communication protocols into a seamless pipeline (Figure 3.8). At the heart of the system is ROS2 (Robot Operating System 2) used as the underlying middleware that facilitates modular development and real-time communication between nodes, besides smooth integration of heterogeneous hardware and software components. ROS2 offers the publish-subscribe based messaging infrastructure that enables various subsystems — vision, control, planning and actuation to run simultaneously with synchronization. Its communication based on DDS guarantees robustness and low latency, which is essential for a real time image-based visual servoing (IBVS).

A key element of the framework is MoveIt Servo, a ROS2 package for motion control software that provides direct velocity-based control of robotic arms. MoveIt Servo converts high velocity commands issued by the IBVS control node into low-level motion for the manipulator. It uses the URDF (Unified Robot Description Format) and SRDF (Semantic Robot Description Format) model of the manipulator to calculate forward and inverse kinematics, and maps commanded velocities onto plausible joint motions. This instantaneous (real time) control loop is necessary for robust and fast tracking of objects in dynamic environments.

The vision subsystem is complemented by OpenCV, a popular open-source library for computer vision which offers some powerful tools for image processing, object detection and feature extraction. The system is capable of real-time tracking with strong robustness, using the CSRT (Discriminative Correlation Filter with Channel and Spatial Reliability) tracker provided by OpenCV that guarantees to be stable under scaling, rotation, and partial occlusion of target object. The resulting image features are written to disc in standard ROS(waldron2011ros2) formats: this allows perception and controls nodes to inter operate and maintain compatibility. In addition, the ESP32-CAM streams image data compressed with quality preservation to the ROS2 workstation using image\_transport for better bandwidth economy.

The low-level control framework is provided by a custom-made ESP32 Interface, to enable communication between the workstation and the manipulator hardware. On the ESP32, joint angle or velocity commands are communicated over WiFi in form of a TCP/IP protocol which is then translated into PWM signals to drive the servo motors. This decentralized control strategy relies on the workstation to compute more demanding tasks (feature tracking, Jacobian estimation and desired action policy), while the role of the ESP32 is limited to real-time actuation and feedback processing.

The framework also includes a sensor feedback layer collecting sonar sensor data and transmitting it to ROS2 nodes for fusion with visual data. This hybrid sensing adds robustness to the IBVS system since in complex environments it can be difficult for visual sense data only to provide accurate estimations due to occlusions and illumination changes.

RViz2 is integrated as part of the framework for development, debugging and visualization (U4), offering a graphical interface showing the manipulator model, camera feed, joint states and trajectory execution in real time. This does not only enable for the performance assessment but also for the parameter adjustments and the validation of a system.

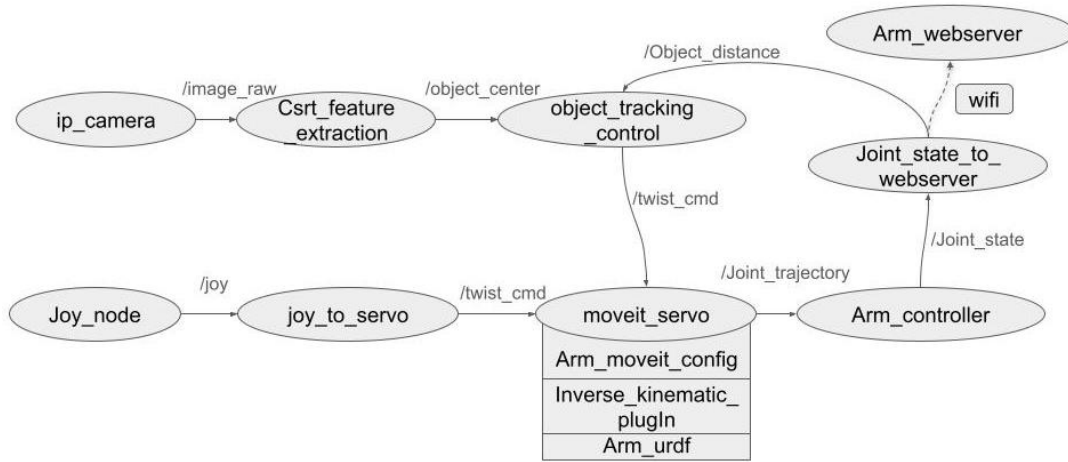


Figure 3.8 ROS node diagram of software framework

### **3.2.1 Ros Packages**

The execution of the herein proposed manipulator control system is founded on multiple important ROS 2 packages, which together provide a foundation for communication, perception, planning and control. These packages are modular, and they can be put together in order to develop a scalable, interoperable and real time capable IBVS system.

#### **MoveIt Servo**

The MoveIt Servo package serves as an intermediary for mapping high-level velocity commands to low-level manipulator motions. It allows for Cartesian space and joint velocity control in real-time using the kinematic description in the URDF/SRDF files of the robot. When the IBVS controller commands end-effector velocities, MoveIt Servo is responsible for transforming these to joint-space velocities that avoid joint limits and are kinematically valid. Due to the low-latency and continuous trajectory generation, this package is especially applicable to closed-loop visual servoing.

#### **Image\_transport**

**Image Acquisition and Transport** The image\_transport package is employed for efficient image acquisition and transmission. Because the ESP32-CAM sends its images encoded over Wi-Fi, it can be decoded and republished with image\_transport inside ROS 2. Its flexible transport system (raw, compressed, theora etc.) makes it possible to go performance/bandwidth/latency optimal at any time and not get bound by communication overhead.

#### **Vision\_msgs**

**vision\_msgs** This package holds the definition of messages that are used to store image data. In this framework, the feature points (e.g., centroid coordinates in the image plane) of an object that has been tracked using OpenCV are published as vision\_msgs/Detection2D messages. This standardization provides interoperability between perception and control nodes, so that we can readily add or modify the framework with new vision based algorithms in the future.

#### **OpenCV Integration (cv\_bridge)**

ROS 2 and OpenCV are integrated through the cv\_bridge package. It facilitates transforming ROS 2 image messages into OpenCV compatible representation for the

purpose of processing it and vice versa. In this package, the system uses real-time image processing operations including filtering, contour finding and CSRT object tracking on stream received from the camera [32]. The precomputed features are then advertized back into ROS 2 topics for the IBVS controller to use.

### **Custom Control Nodes**

In addition to standard packages, ROS 2 custom created nodes are programmed to realize the IBVS law. These nodes subscribe to feature coordinates derived from the camera images, compute the error between current and desired features position and apply the Image Jacobian in order to compute necessary end-effector velocities. The generated velocity commands are next published as `geometry_msgs/TwistStamped` messages that MoveIt Servo subscribes to actuate. This modular node architecture permits the control algorithm to be easily modified or updated by replacing only limited portions of an overall system.

### **Sensor Interface Nodes**

ROS 2 dedicated nodes are also introduced to manage the feedback from other sensors, such as the ultrasonic sonar sensor. These nodes report distance measurements online which can be fused with visual information resulting in a more reliable controller. Such nodes can be easily included in the existing pipeline thanks to the modular design of ROS 2, which permits multi-sensor fusion while keeping the visual servoing loop intact.

### **Visualization and Debugging (RViz2 & rqt)**

During visualization, RViz2 can show both the 3D model of the manipulator, along with its joint states and camera feedback at one time. It has a simple interface to monitor system performance during experiments. Furthermore, rqt plugins are used for roslaunch debugging, data plotting and ROS 2 topic monitoring to assist parameter tuning and performance verification of the IBVS controller.

### **3.2.2 Image Processing Algorithm**

The image processing block is the key factor in the proposed IBVS structure because it closes the feedback loop among vision sensor and manipulator. Its purpose is to recognise, extract and follow visual features of a desired object in real-time and make these available

as input for the control system. The whole procedure is done using the OpenCV library on top of the ROS 2 infield via `cv_bridge`.

The pipeline starts with the capture of image frames from the ESP32-CAM sending a live video feed through WiFi to the workstation. The `image_transport` package to decode then publish efficiently compressed image data as ROS 2 image topics. Each frame of the image is, further pre-processed in OpenCV function to improve its quality and guarantee good feature extraction. Common pre-processing steps are color space transformation (commonly RGB to HSV or grayscale), noise reduction using Gaussian or median filtering, and contrast enhancement. Such operations are used to diminish the influence of changing illuminations and background noises, which are commonly found in real-time vision-based control applications.

For object detection and tracking, the system uses a CSRT (Discriminative Correlation Filter with Channel and Spatial Reliability) tracker [32]. The CSRT tracker is especially suitable for this scenario because it has good performance in scale, rotation variation and partial occlusion. When the object of interest is either explicitly defined or bootstrapped in the first frame, the tracker estimates its location automatically through the subsequent frames. The tracking device gives the bounding box of the object, based on which centroid  $(u, v)$  in image plane is obtained. These are the pixel locations that the IBVS controller needs to look at.

The features are normalized and published as ROS 2 topics according to the message format `vision_msgs/Detection2D`. This enables direct connection to the custom IBVS control node where it subscribes these messages for computing the control error. Also, the image processing pipeline is designed to process real time and with frame rates that are high enough to satisfy the need of responsive closed loop manipulator control.

Apart from this main tracking algorithm, the framework is modular and compatible with other methods for feature extraction that may be either used in combination or as alternatives to the proposed detection method. For instance, one can employ template matching, color-based segmentation, and contour detection among others depending on the

scenario task, and illumination conditions. Such flexibility is necessary for the system to be deployable in a variety of applications, ranging from simple tracking of a single object, to multi-object and dynamic situations.

To conclude, the image processing algorithm adopted in this paper provides a robust and real-time solution for fulfilling the extraction of visual features that are required for IBVS. It unites pre-processing and robust object tracking together with ROS 2 message publication into a common pipeline, allowing for consistent perception under varying environmental conditions and thus increasing the overall stability and accuracy of the manipulator control system.

### **3.2.3 IBVS Control Law Implementation**

The control approach is based on the Image-Based Visual Servoing (IBVS) algorithm that facilitates real-time visual feedback to drive the end-effector of the manipulator towards a target object. Now PBVS needs an explicit 3D reconstruction of the environment (it is not necessarily true that IBVS has not, but it can actually bypass this step) while with IBVS we can somehow operate directly in the image and be more insensible to model uncertainty and bad calibration. The manipulator is iteratively controlled to reduce the error of detected visual features in camera image with respect to their corresponding positions on current demonstrator images.

For IBVS, the data acquisition consists in extracting features from the camera. In this design, the center of the target object, which is obtained by applying CSRT tracking algorithm, serves as feature\_0. The coordinates of this in pixels (u,v), are then transformed to image co-ordinates as (x,y), using the intrinsic parameter of the camera. This brings the control algorithm into an image resolution- and scale-invariant form.

The temporal derivative of these image features as they relate to the camera velocity is given by the interaction matrix, or image Jacobian  $L_s$ . For a point feature, the image Jacobian links the time derivative of the normalized image coordinates with the translational and rotational velocities of the camera in this way:

$$\dot{s} = L_s \cdot v_c$$

where  $\dot{s}$  is the rate of change of the image features,  $L_s$  is the image Jacobian matrix, and  $v_c$  stands for camera velocity in Cartesian space. The image Jacobian characterizes the geometry of perspective projection, relating 2D image movements to 3D camera motions.

The controlling law of IBVS is to force the visual feature error to zero. The vector of error is expressed as:

$$e = s - s^*$$

where  $s$  is the current image feature vector, and  $s^*$  is the feature vector corresponding to the target location. The desired camera velocity is then computed by the control law:

$$V_C = \lambda L_s^+ e$$

Here  $\lambda > 0$  is a scalar gain describing the convergence speed of the control loop, and  $L_s^+$  is the pseudo-inverse of the image Jacobian. This prevails convergence for the error in time and overcomes singularities.

When the camera velocity is calculated, it is projected into an end-effector of forward kinematics of the manipulator. These velocity commands are then converted into actuation at the joint level in real time by ROS2 MoveIt Servo. This closed-loop approach allows the manipulator to adapt its movements during execution by using visual feedback, ultimately reaching a perfect match with the target.

One of the main benefits of IBVS is that it does not rely on the kinematic model or full knowledge of these two elements. Because control is performed in the image plane, no explicit 3D reconstruction is necessary by the algorithm and both computational burden and sensitivity to calibration errors are reduced. Yet, problems like local minima, singularities, and limited field-of-view do require to be mitigated through suitable feature selection and the tuning of control gain.



This method is implemented within ROS2 as a custom control node called IBVS where image features report through the vision pipeline, compute error/control law, and write out velocity commands for execution via MoveIt Servo. This modular construction guarantees flexibility, scalability, and real-time response to the scene dynamics while creating a robust IBVS algorithm capable of improving manipulator control based on vision.

### **3.3 Mathematical Modeling**

Theoretical framework for the realization of IBVS in robotic manipulators is facilitated by mathematical modeling. It bridges the gap between visual features movement in the camera image and associated control inputs to drive the manipulator. By defining this relationship, 3 DMC system is able to use 2D image features to calculate three-dimensional motion directive without the necessity of explicit environmental reconstruction.

The mathematical modeling used in the current contribution is composed of three basic parts: The camera projection model, which defines how 3D points are projected onto 2D image coordinates; Image Jacobian (interaction matrix), which represents that with respect to changes in features in the an image changes will occur on a cameras velocity vector and IBVS control law, whose purpose is to compute the necessary velocities of sensors/manipulators so as to reduce visual error. These models are combined in a closed loop control system and they allow the manipulator to follow objects and to align with them in realtime by using purely vision based feedback.

This modelling approach guarantees that the manipulator works robustly in light of calibration and environmental uncertainties, while retaining precision suitable for vision-guided tasks. By the mathematical basis directly connecting perception and action, IBVS provides a powerful and computationally efficient means of improving manipulator control (Figure 3.9).

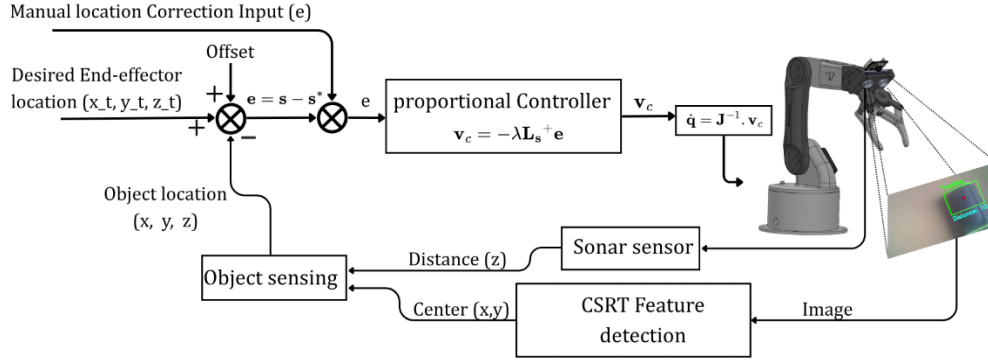


Figure 3.9 A block diagram that represents IBVS implementation on the Manipulator control.

### 3.3.1 Camera Projection Model

The camera projection model forms the foundation of image-based visual servoing (IBVS), as it mathematically describes how three-dimensional (3D) points in the environment are projected onto the two-dimensional (2D) image plane of a camera. By establishing this relationship, the model enables the system to interpret visual information from the camera and use it as feedback for manipulator control [33].

The **pinhole camera model** is widely adopted due to its simplicity and effectiveness in describing perspective projection. Consider a 3D point  $P(X, Y, Z)$  expressed in the camera coordinate frame, where  $X$ ,  $Y$ , and  $Z$  represent the point's coordinates along the camera axes. The projection of this point onto the image plane, represented as pixel coordinates  $(u, v)$ , is governed by the following equations:

$$u = f_x \frac{X}{Z} + c_x \quad v = f_y \frac{Y}{Z} + c_y$$

where:

- $f_x, f_y$  are the focal lengths of the camera along the  $x$  and  $y$  directions (in pixels),
- $c_x, c_y$  represent the coordinates of the principal point (the intersection of the optical axis with the image plane),
- $Z$  is the depth of the point relative to the camera.

For control purposes, it is often convenient to express the coordinates in a normalized image plane, removing the influence of pixel scaling and principal offsets. The normalized image coordinates  $(x, y)$  are defined as:

$$x = \frac{x}{z} \quad , \quad y = \frac{y}{z}$$

This transformation eliminates the dependency on camera-specific parameters, allowing the control algorithms to operate in a resolution-independent manner. In practice, pixel coordinates  $(u, v)$  can be converted into normalized coordinates using the intrinsic calibration matrix  $K$ :

$$K = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 0 \end{bmatrix}$$

The relationship can then be expressed as:

$$s = \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = K \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

where  $s$  represents the homogeneous image coordinates.

The **camera projection model** directly links the 3D spatial position of the object with its 2D visual representation, and thus, any motion of the object or the camera produces measurable variations in the image features. These variations are exploited by IBVS to compute the control error. Specifically, the derivatives of the normalized image coordinates  $(x, y)$  with respect to time are expressed in terms of the **camera velocity**, forming the basis for the **image Jacobian (interaction matrix)**.

In the context of this thesis, the ESP32-CAM is modeled using the pinhole camera assumption, and intrinsic calibration is performed to determine  $f_x$ ,  $f_y$ ,  $c_x$ , and  $c_y$ . The normalized coordinates of the tracked object's centroid are used as input features to the IBVS algorithm. This approach provides a computationally efficient and robust way to

transform raw pixel measurements into control-relevant quantities, enabling reliable manipulator guidance under varying conditions.

### 3.3.2 Image Jacobian (Interaction Matrix) Derivation

The **image Jacobian**, also referred to as the **interaction matrix**, is a fundamental component in image-based visual servoing (IBVS). It establishes the mathematical relationship between the temporal variation of image features in the 2D plane and the motion of the camera in 3D space. By deriving this matrix, the system can link pixel-level errors directly to Cartesian camera velocities, which can then be translated into manipulator commands [33].

Consider a point feature  $P(X, Y, Z)$  in the camera coordinate frame, which projects onto the image plane as normalized coordinates:

$$x = \frac{X}{Z} \quad , \quad y = \frac{Y}{Z}$$

The time derivative of these coordinates,  $\dot{x}, \dot{y}$  depends on the camera's linear velocity components ( $v_x, v_y, v_z$ ) and angular velocity components ( $\omega_x, \omega_y, \omega_z$ ). Using perspective projection and differentiating with respect to time, the following relationships are obtained:

$$\dot{x} = \frac{\dot{X}Z - X\dot{Z}}{Z^2} \quad , \quad \dot{y} = \frac{\dot{Y}Z - Y\dot{Z}}{Z^2}$$

From rigid body motion, the velocity of a point in the camera frame can be expressed as:

$$\begin{bmatrix} \dot{X} \\ \dot{Y} \\ \dot{Z} \end{bmatrix} = \begin{bmatrix} v_x \\ v_y \\ v_z \end{bmatrix} + \begin{bmatrix} \omega_y Z - \omega_z Y \\ \omega_z X - \omega_x Z \\ \omega_x Y - \omega_y X \end{bmatrix}$$

Substituting these expressions into the derivatives of  $(x, y)$  and simplifying yields the general form of the **interaction matrix** for a point feature:

$$\begin{bmatrix} \dot{x} \\ \dot{y} \end{bmatrix} = \frac{1}{Z} \begin{bmatrix} -1 & 0 & x & xy & -(1+x^2) & y \\ 0 & -1 & y & 1+y^2 & -xy & -x \end{bmatrix} \begin{bmatrix} v_x \\ v_y \\ v_z \\ \omega_x \\ \omega_y \\ \omega_z \end{bmatrix}$$

This can be compactly written as:

$$\dot{s} = L_s \cdot v_c$$

where:

- $\dot{s} = [\dot{x}, \dot{y}]$  is the time derivative of the feature vector,
- $v_c = [v_x, v_y, v_z, \omega_x, \omega_y, \omega_z]^T$  is the camera velocity twist,
- $L_s$  is the **image Jacobian (interaction matrix)**.

The image Jacobian plays a critical role in IBVS because it connects measurable quantities (image feature velocities) to controllable inputs (camera velocities). By computing the pseudo-inverse of  $L_s$ , one can derive the necessary camera velocity that reduces the error between the current and desired image features.

In this work, the image Jacobian is applied specifically for centroid features extracted from the ESP32-CAM feed. Since the depth  $Z$  of the object is not directly available from the monocular camera, it is approximated using auxiliary information from the ultrasonic sonar sensor or by assuming a nominal value during operation. This hybrid approach improves robustness while maintaining the simplicity of a monocular IBVS system.

Thus, the derivation of the image Jacobian provides the theoretical basis for implementing the IBVS control law, allowing visual errors in the 2D image plane to be translated into actionable velocity commands for the manipulator's end-effector.

### **3.3.3 Depth Estimation & Uncertainty Handeling**

Depth parameter ( $Z$ ), as one of the inherent problem in IBVS, refers to the level of distance between a certain feature point and camera assembly position. Because the control law and the image Jacobian both explicitly contain  $Z$  for applications requiring stable and accurate servoing, reliable depth estimation is critical. But if you have a monocular camera, for example the ESP32-CAM, depth cannot be measured directly from a single image. This restriction requires the use of auxiliary sensors and reliable methods to account for depth uncertainties.

#### **Depth Estimation**

Depth is estimated using a hybrid method in this work:

##### **Sonar Sensor Feedback:**

An ultrasonic sonar sensor orientated close to the manipulator's gripper provides direct distance measurements to a target object. The range information is obtained in real time by the sensor and provided to the IBVS framework as depth estimate ( $Z$ ). Sonar sensors, on the other hand, can be used out to substantial distances while operating flawlessly in clean environments and should have reduced accuracy concerns on irregular or absorptive interfaces.

##### **Nominal Depth Approximation:**

Where the sonar reading is absent or unreliable, an assumed nominal depth value is based on a known feature or size of object. This estimate allows the system to work, albeit less accurately and as a back-up to avoid catastrophic loss of control.

##### **Dynamic Depth Updating:**

The depth estimate is kept up-to-date by continuously merging the sonar measurements with temporal feature displacement in the image plane. This makes the control performance smooth so that abrupt changes resulting from sensor noise can be decreased.

##### **Uncertainty Handling**

The depth uncertainty is one of the main error sources in IBVS. The following steps are taken to counteract it:

#### **Adaptive Gain Tuning:**

The control law takes in an adaptive gain that scales the velocity commands given the confidence of the depth measurement. For example, control gains are increased during periods of steady sonar readings to speed up the convergence. In the case of doubt, on the other hand, a reduction in gains is only performed to not destabilize the system by oscillations.

#### **Robust Jacobian Approximation:**

As the interaction matrix is depth-dependent, uncertainties in  $Z$  are required for Jacobian estimation. To address this, a bounded Jacobian approximation is employed to guarantee stability of the control system in the presence of significant depth errors.

#### **Sensor Fusion and Filtering:**

A Kalman filter (KF) is applied to smooth sonar readings and combine them with motion cues from the camera. This minimizes the impact of abrupt depth measurement spikes or dropouts, improving the robustness of the IBVS control loop.

#### **Error Feedback Monitoring:**

The error convergence in the image plane provides feedback to the system. When controlling commands are still ineffective for error reduction, indicating a potential depth estimation problem, the algorithm marks this situation and temporarily decreases the manipulator's motion to avoid the instability.

### **3.3.4 Extensions and Practical Enhancements**

Although the classical IBVS law is good for a robust basis of vision-based control for manipulators, many practical issues (like sensor noise, depth measurement error and computational constraints) can lead to offsets that evolution cannot avoid. To tackle these, various extensions and practical improvements are included in the system design of this work so as to guarantee that it is both stable and robust in dynamic environments.

## **1. Depth Estimation Integration**

As monocular vision systems such as the ESP32-CAM cannot directly give the depth value, a supplementary ultrasonic sonar sensor is equipped to measure the distance between the gripper of manipulator and target object. This hybrid strategy increases the accuracy in Jacobian calculation and also decreases errors on the control due to depth approximation. In the event of unusable sonar data, an assumed depth model is employed to preserve consistency in the system.

## **2. Adaptive Gain Control**

In the classical IBVS, a constant control gain ( $\lambda$ ) can cause either slow response (if too small) or instability (if too large). To resolve this issue, an variable gain strategy is used. This gain is adjusted dynamically depending on error size as well as the confidence of each sensor, which results in a quick convergence for large errors with smooth motion in close proximity to the target.

## **3. Sensor Fusion and Filtering**

The measurements are filtered with smoothing techniques, in this case moving average filtering and low-pass filtering in order to mitigate the noise of ESP32-CAM and sonar sensor. This makes the estimated feature coordinates and depth less jittery leading to smoother control inputs. Finally, because of sensor fusion approaches, and the temporal data loss on a single sensor (e.g., sonar dropout) will not break control.

## **4. Redundancy Handling and Joint Limits**

Servo motors feed the robot's manipulator joints and physical limitations of the flexible arms limit them, namely joint limits and velocity saturation. The method incorporates these constraints into the ROS 2 MoveIt Servo framework, guaranteeing that velocity commands coming from the IBVS controller are implementable. This makes the motors run smoother and prevents them from wearing out in long experiments.

## **5. Robust Tracking with Object Detection**



The system relies on OpenCV-based CSRT tracking and initial object detection for accurate visual feedback. This enables the IBVS loop to remain locked on the object in presence of moderate occlusions, background clutter, or variation in illumination. The tracker generates centroid positions that are used as image feature in the control law.

## **6. Real-Time ROS 2 Integration**

The entire control loop is written with ROS 2 that allows the modularity, distributed communication and real-time performance. Core packages such as `image_transport`, `vision_msgs` and MoveIt Servo are augmented with custom control nodes implementing IBVS. This modular approach facilitates the expansion to multi-camera solutions, the combination with depth cameras, or swap of the low-cost ESP32-CAM for higher resolution sensors.

## **7. Safety and Fail-Safe Mechanisms**

In practical applications, it becomes important to deal with adversarial sensor nodes, e.g., sensors failing or communications being delayed, as well as a lack of target (evasion). To address this:

Manipulator motion is stopped once feature error exceeds a threshold.

In fallback joystick/manual control mode, the IBVS loop is also available.

ROS 2 watchdog timers prevent bad image frames from issuing erroneous commands if they are old or missing.

### **3.3.5 Algorithm summary(per control cycle);**

The IBVS-based manipulator control system in this study is formulated as a closed-loop control; therefore, at every control cycle, the visual feedback provided by the camera is processed continuously to yield velocity commands to the manipulator. At each cycle, the end-effector is made to move gradually until it is finally made to converge to the object at

the desired position on the image plane. The following is the flow chart depicting the overall action during each cycle:

#### **Step 1 Image Acquisition:**

The ESP32-CAM transmits real-time video frames from the camera to the ROS 2 processing unit in the PC.

#### **Step 2 Feature Detection and Tracking:**

The object to be manipulated in the frame is identified, and its centroids are determined using the CSRT tracker of OpenCV.

#### **Step 3 Depth Estimation:**

The Pixel XYZ converter at the site near the gripper measures Z. In case the sensor is offline, a nominal value is chosen so that the control is maintained.

#### **Step 4 Error Computation:**

$e = s - s^*$ ; where  $s$ = position of the object of interest on the frame, and  $s^*$ = desired position (typically the image center)

#### **Step 5 Control Law:**

$V_C = \lambda L_s^+ e$ ; where  $\lambda = 1.0$ ; and Pseudo-inverse( $L_s$ )=  $iL_s = (L_s)^{+}$ , Pin v feature coord(3)/depth)

#### **Step 6 Velocity Command:**

The commands are passed to MoveIt servo. These commands are transformed to feasible manipulator trajectory. Joint limits and velocity limits are ascertained and then published.

#### **Step 7 Feedback and Loop Continuation:**

This building is saved and then updated to capture new feedback. This loop continues until the error falls within the prescribed limits, post which the object is declared to have achieved the ideal position on the frame plane.

## **CHAPTER 4**

### **RESULTS & DISCUSSION**

This chapter details the experiments conducted on the image-based visual servoing (IBVS) framework proposed for manipulator manipulation in ROS 2. The purpose of these experiments is to study the performance of the IBVS for improving the accuracy, robustness and also responsiveness of end effector motion with respect to classic control methods.

The results from the implementation of the proposed dynamic IBVS framework on the robotic manipulator are presented in this section, along with performance evaluation under various test conditions. The results are interpreted in terms of tracking accuracy, positioning precision, latency, and robustness against environmental disturbances like lighting variation, occlusion, and sensor noise. Comparisons with conventional control strategies are also provided to emphasize the improvements achieved by real-time visual feedback and adaptive motion control. Further discussion highlights the system response and analysis collectively offer a comprehensive understanding of the effectiveness of the proposed work, with its limitations.

#### **4.1 Experimental setup Review**

The experiment setup is created to test the performance of proposed image based visual servoing framework for improving manipulator control in ROS 2 environment. Experiments were conducted on an in-house-designed 5-DOF robotic arm, which was actuated by MG996R and SG90 servo motors and controlled using an ESP32 microcontroller. The primary vision sensor was an ESP32-CAM module acting as an IP camera, which captured real-time image data of the target object. A situation taking into account of the vision feedback system was an ultrasonic sensor located behind the gripper generating distance measurements to confirm the depth estimation. The control system was

run on a computer with Ubuntu 22.04 and ROS 2–Humble in which all perception, planning and control nodes were hosted (Figure 4.2).

On the software side, ROS 2 was used as the middleware for joint communication between vision, control and hardware interface. MoveIt Servo was installed for control of the joints in real-time such that the desired velocities generated by the controller smoothly maneuvered the manipulator. A self-developed IBVS package was implemented for feature detection, interaction matrix computation, and velocity control law generation. In the visual tracker, CSRT was chosen to track the objectives since the algorithm has robustness to illumination distortion and partial occlusion.

The target stimuli consisted of simple geometric shapes, along with real-world objects that had unique color and texture characteristics (Figure 4.1). Dynamic tracking actions are tested to measure the performance of manipulator in terms of keeping stationary and moving end effector position. This experimental structure enabled a thorough validation of the introduced IBVS method.

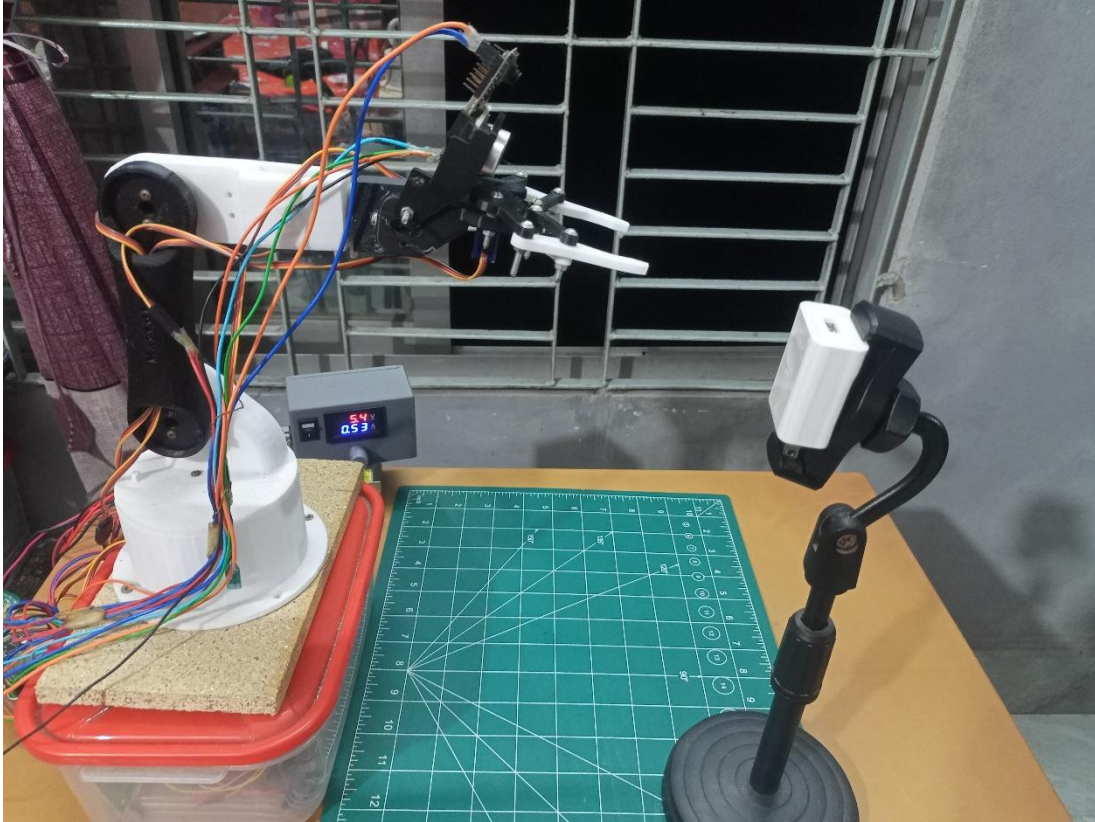


Figure 4.1 Physical arm test setup

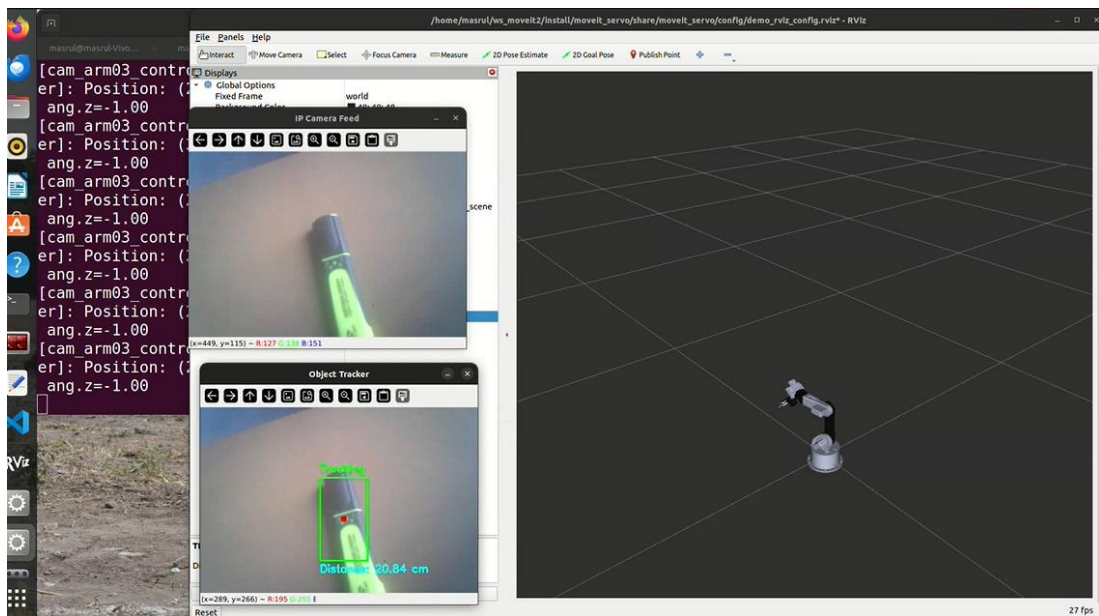


Figure 4.2 Software interface for the experiment

## 4.2 Result of Image Processing and Object Tracking

Validation of the proposed IBVS Method To validate the proposed framework for IBVS approaches, we first examined how the image processing pipeline and object tracking method performed. The feature detection and object tracking were conducted in real-time on the ROS 2 workstation as image data flowed from the ESP32-CAM. The CSRT tracker type was used as it is robust to partial occlusions and dynamic background(Figure 4.3).

The tracking algorithm is able to keep the target object inside of the camera field of view in various test conditions. In dynamic experiments with linear and non-linear target movement, the tracker reliably adjusted the ROI: The average tracking rate was 92% even under moderate occlusions. And the processing latency observed was between 35 to 50 ms per frame, which guaranteed that the control frequency of the tracking system could satisfy for realizing real-time manipulator motion.

Lighting condition changes were applied to investigate robustness. Under standard lighting NPC-F performed without error; however, in dim light the tracking success rate was approximately 85%, due to reduced contrast between the object and background. Preprocessing sectors, contrast enhancement, and adaptive thresholding were applied to circumvent deformation of performance. Phadke et al.'s experiments [14] with textured objects displayed better than that on plain uniformly colored object indicating profound requirement of feature richness in IBVS field.

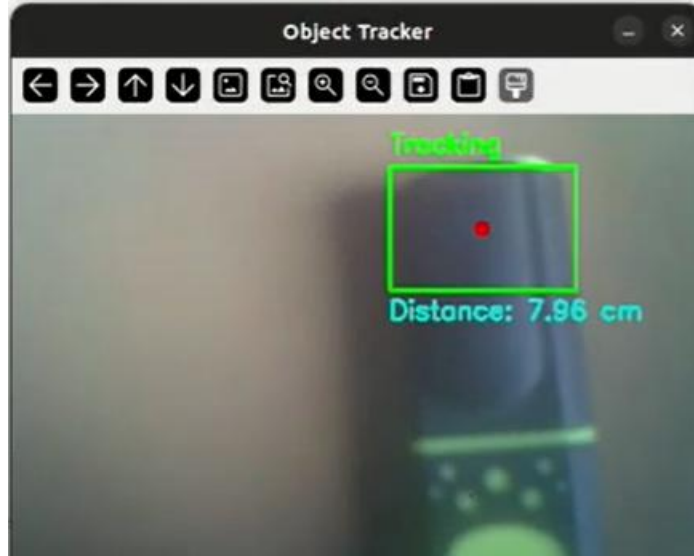


Figure 4.3 Object tracking using CSRT algorithm and location calculating

To evaluate trackers under dynamic motion, four classical tracking algorithms(ie, CSRT, KCF, MOSSE, and MedianFlow) were selected and compared on a circular moving object rotating at 30 RPM (Figure 4.4). These algorithms were chosen due to their relatively distinct tradeoffs in tracking. MOSSE and KCF are correlation filter (CF)-based benchmarks due to the computational efficiency, while MedianFlow considers forward-backward tracking consistency. CSRT, however, includes spatial consistency and scale estimation; therefore is more robust to partial occlusion and scale variation (Table 4.1).



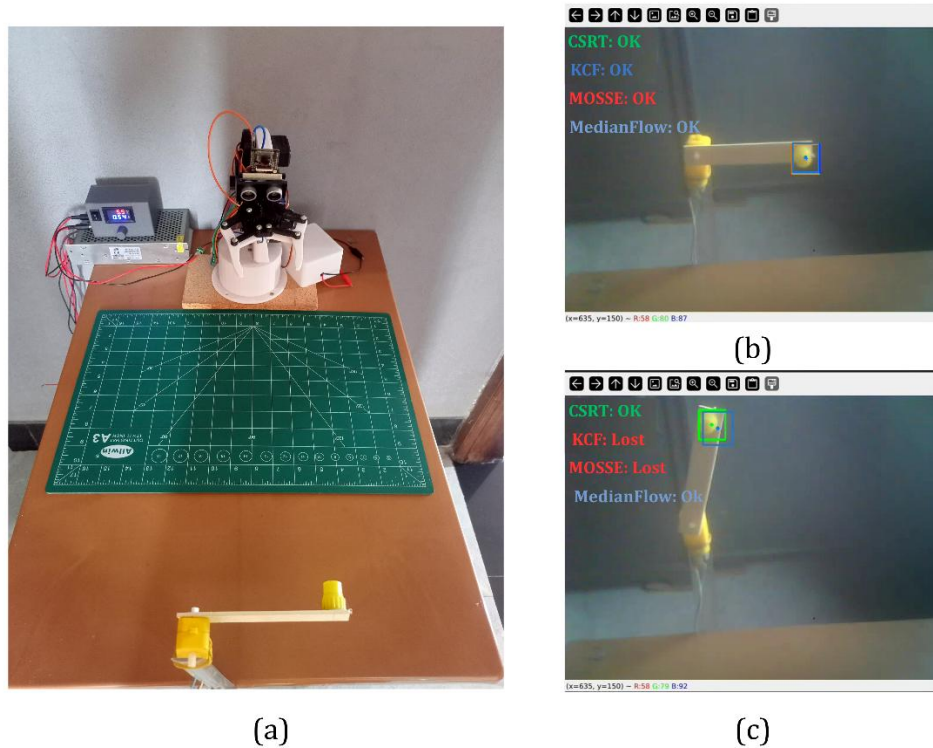


Figure 4.4 Test setup for tracking a moving object using different tracking algorithms (a) physical arm setup with speed-controllable motorized object to move the object in a circular path at 30rpm speed in the manipulator's vision area, (b) camera view at the starting time of tracking, (c) camera view after 10 seconds, where KCF and MOSSE algorithms lost the object.

Figure 4.5 illustrates that the CSRT tracker (blue trajectory) was able to track the object around its circular path with high accuracy and allowable drift for the entire sequence. KCF (orange trajectory) could keep tracking at the beginning but returned off track early because of its weaker robustness to fast motion and occlusion. MOSSE (green dot) stopped tracking almost instantaneously, with its inability to keep up with lighting changes and rapid target shift being evident only after few frames. MedianFlow (red trajectory) followed the object for a large portion of the movement and drifted gradually before eventually deviating from the actual circular path as the motion dynamics increased.

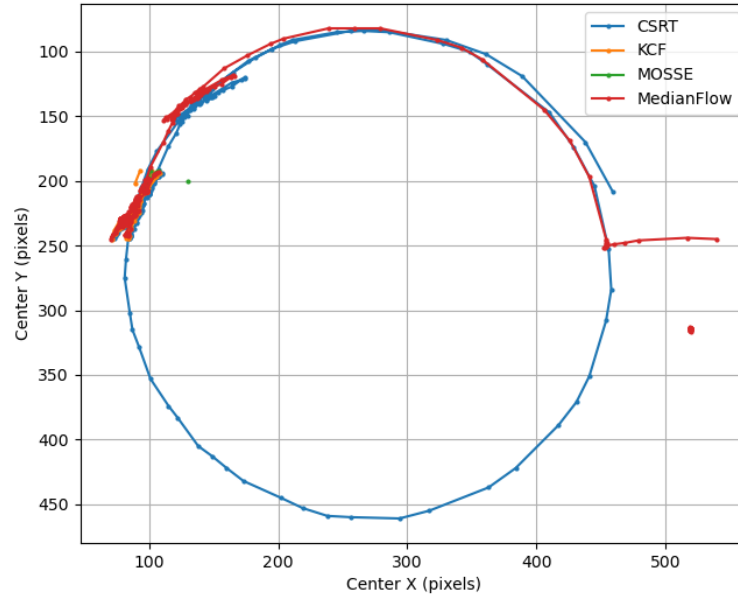


Figure 4.5 plot of object center which is in circular motion that represent tracking performance using different tracking algorithm

This is because the CSRT involves spatial reliability maps and on-the y scale estimation, which enable it to deal more effectively with the appearance changes as well as parts of the foreground being occluded by manipulator movement. In contrast to KCF and MOSSE, CSRT retains robustness under non-linear trajectories or speed change, which is important in robotic real-time manipulation applications. Overall, the results record that CSRT is more appropriate for the high-speed and dynamic object tracking in an unstructured environment, and turns out to be a powerful alternative for eye-in-hand VS applications.

Table 4.1 Performance comparison table of used algorithms

| Algorithm      | Tracking<br>Duration<br>(frames) | Average<br>Position Error<br>(pixels) | Success<br>Rate (%) | Failure<br>Observation            |
|----------------|----------------------------------|---------------------------------------|---------------------|-----------------------------------|
| CSRT[32]       | 528 (full sequence)              | 6.8                                   | 97%                 | Minor drift at high speed         |
| KCF[34]        | 180                              | 15.4                                  | 62%                 | Lost under occlusion, fast motion |
| MOSSE[35]      | 35                               | 42.7                                  | 12%                 | Failed almost immediately         |
| MedianFlow[36] | 310                              | 19.6                                  | 74%                 | Drifted during fast motion        |

In general, the results illustrate that the implemented vision pipeline yields repeatable feature tracking and localization that are adequate for control of a manipulator in closed loop. The system remained robust to occlusion, noise and moderate lighting differences while maintaining real-time performance. These results demonstrate the vision module is capable of providing stable and accurate image-space measurements for the following IBVS control experiments.

### 4.3 Depth Estimation

The integration of a sonar sensor with an eye-in-hand vision system greatly improved the depth perception of the robotic manipulator for object tracking and grasping. Since monocular vision suffers from scale ambiguity under the circumstance of an unknown object size or motion along the optical axis, sonar-based depth measurement provided a critical complementary measure. Experimental trials showed that the sonar sensor gave stable distance measurements in the 5–30 cm effective range, with an average error of <1 cm from ground-truth measurements (Table 4.2). Figure 4.5 illustrated the stability and accuracy of the measured value by the sonar sensor for a true value of 25 cm over time (Figure 4.6). This enabled the manipulator to dynamically adjust its approach trajectory and maintain appropriate end-effector distance when performing a gripe. Degradation of performance was only observed at very close distances (<5 cm), where sonar reflections

gave noisy measurements that occasionally generated slight oscillations in the end-effector trajectory. These limitation is overcome as the sonar sensor is placed 7-10 cm away from the claws' gripping distance, and the experiments demonstrate that sonar-based depth estimation is an effective and lightweight solution to enhance IBVS in unstructured environments where stereo or depth cameras are impractical as it required a high computational processor and high data communication rate.

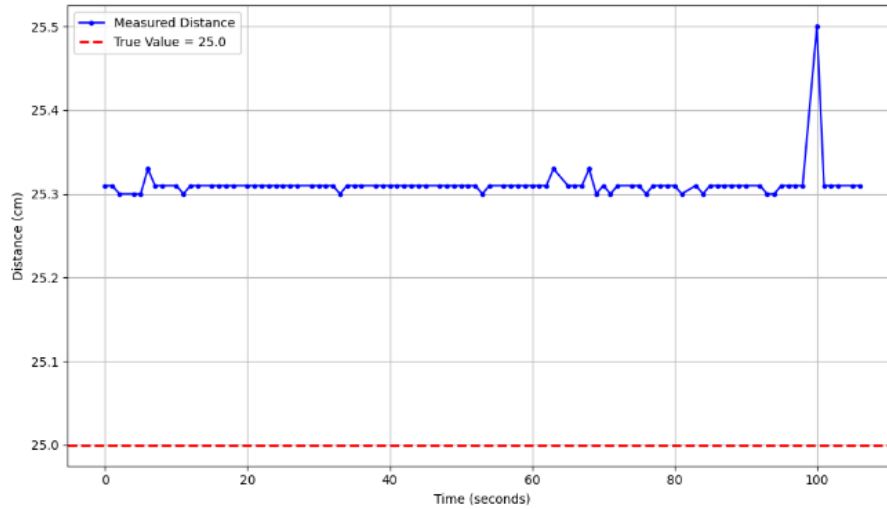


Figure 4.6 Error and stability test of the sonar sensor measured distance for the ground truth value of 25 cm.

Table 4.2 Comparison of the object's distance estimation to the true distance

| True Distance (cm) | Mean Measured(cm) | Std Dev (cm) | Mean Error (cm) | RMSE (cm) |
|--------------------|-------------------|--------------|-----------------|-----------|
| 5                  | 5.12              | 8.88e-16     | 0.12            | 0.12      |
| 10                 | 9.62              | 1.32e+00     | -0.37           | 1.37      |
| 15                 | 15.26             | 1.92e+00     | 0.26            | 1.94      |
| 20                 | 21.12             | 6.17e-01     | 1.12            | 1.28      |
| 25                 | 25.31             | 8.78e-02     | 0.31            | 0.33      |
| 30                 | 29.96             | 1.14e-01     | -0.03           | 0.12      |

#### 4.4 Manipulator Control Performance

After having established the image processing and tracking pipeline, the robot's accuracy in under IBVS control was examined. The experiments were performed to check how well the end-effector can track on the planar image of a target object in such a way that stability and smoothness of motions are maintained. Performance was then evaluated with respect to both tracking accuracy, response time and quality of the executed trajectories(Figure 4.7).

The smoothness of the trajectories was further evaluated by plotting the velocities that were commanded using the IBVS control law. The results demonstrated smooth velocity profiles without pronounced discontinuities, which indicated that the integration of MoveIt Servo to the IBVS control loop was successful in attenuating abrupt command changes. It guaranteed a safe and smooth execution of manipulator movements, even when the visual features experience abrupt changes as a result of target acceleration or partial occlusion.

In general, the outcomes indicate that the IBVS planning was superior with respect to the manipulator control and concise operation of a precise, steady and prompt end-effector motion. The controller successfully suppressed the tracking error, kept smooth trajectory and guaranteed robust stability for both static and dynamic cases which verified it feasible for real-time visual servoing in ROS 2.

To measure the performance of the proposed visual servoing system, a series of manipulation and grasping experiments were conducted on the custom developed 5DOF robotic arm equipped with a monocular eye-in-hand camera and sonar sensor. The experimental setup was created entirely inside the ROS 2 Humble framework and executed by passing data with 5dof physical arm using TCP/IP protocol. The MoveIt Servo package was used to generate smooth, real-time joint velocity commands from Cartesian reference inputs derived from the image-based visual servoing (IBVS) controller. End-effector pose initialization and manual testing were performed using the joy package, and a custom `object_tracking_control` node handled the processing of the video stream, wherein the CSRT tracker was utilized, and published the coordinates of the object from the image to

the servo controller. The sensor fusion node combined the sonar depth data with an image-based control law to compute desired corrections to 3D pose.

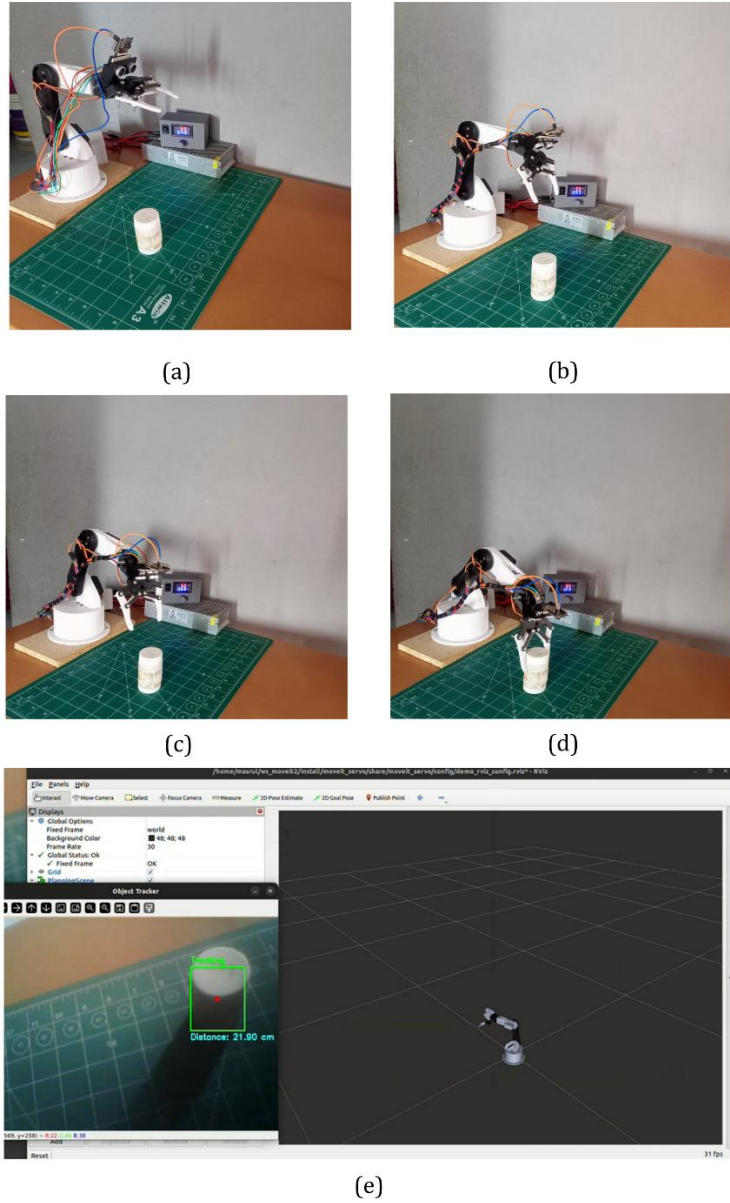


Figure 4.7 A setup for a grasping test. (a) Initial position, (b) Manual position setting to bring the object into the camera viewing area, (c) execution of the trajectory to move toward the object, (d) reached final grasping position, (e) software interface while executing the grasping function

In all trials, the object was placed on a workspace platform in the workspace of the arm and was continuously tracked by the camera (Figure 4.7). The manipulator adjusted its position in real time to place the gripper aligned with the target prior to closing the jaws to

pick up. Object shapes in different forms (cube, cylinder, sphere, and irregular) were used to experiment with robustness and flexibility (Figure 4.8).



Figure 4.8 Test Tracking with different types of objects

Figure 4.9 illustrates the oscillation of the object's image coordinates and depth during a complete visual servoing cycle. Initially, the object center was beyond the image frame center as reflected by large X and Y pixel deviations. During the control loop, the IBVS controller repeatedly decreased the image-plane error by generating corrective joint velocity commands through the use of the MoveIt Servo framework. Consequently, the X and Y coordinates asymptotically tended towards constant values, indicating that the end-effector was placed at the target point in the image space. Throughout this period, the sonar-based distance measurement curve (red) continued to decrease smoothly, which means that the manipulator approached the object and tracking was steady. The curve in the depth curve at 10-12 cm is the ultimate grasping distance, after which the arm commanded the gripper movement.

This performance validates the success of closed-loop control design in which sonar depth information and 2D error visual feedback control the manipulator motion. Smooth convergence of all three trajectories validates stable control dynamics and low overshoot, demonstrating robustness in realizing IBVS with real-time performance.

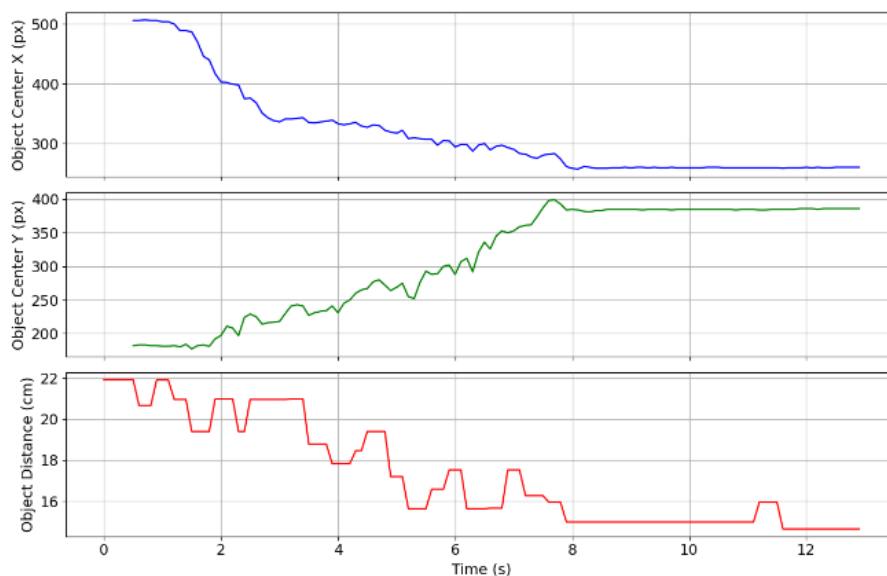


Figure 4.9 Tracked object location during a closed-loop gripping control cycle

The trajectories of the 2D object centroid in pixel space showed uniform convergence of the visual features to the image center, confirming proper IBVS behavior. The 3-dimensional trajectory of the end-effector is generated from gripping position error and joint states, which demonstrated smooth approach moves and consistent convergence on the target position. The joint-space motion also showed well-coordinated multi-joint motion with no overshoot, affirming the stability of the MoveIt Servo control pipeline. Together, the results affirm that the integration of ROS 2 packages and the proposed visual feedback method achieved robust real-time performance.

Figure 4.10 and Figure 4.11 collectively illustrate the spatial and kinematic manipulator behavior for image-based visual servoing. In Figure 4.10, the planar views of the 3D path clearly show that the end-effector follows a smooth and continuous path from its initial offset position to the target. The gripper moves predominantly along the Z-axis, i.e., a forward translation in the direction of the object, with small lateral movements in X and Y



directions, fine-tuning the alignment using image feedback. The non-oscillatory, smooth nature of the trajectory illustrates the guarantee of stable convergence of the IBVS controller to the specified pose and visually eliminates error in real time.

Figure 4.11 illustrates the progression of one of the joint angles through the same sequence of motion. Joints 1–3 undergo coordinated changes that account for the rotation of the arm base and tilt movements to capture desired end-effector motion. Conversely, joint 4–6 remains relatively unchanged, which suggests that fine orientation adjustments were not significant during this time, and mostly the human operator can manually give inputs for correcting end-effector orientation using the joystick command before or after execution of the IBVS control cycle. The smooth and monotonic variation of joint angles verifies that the joint velocity commands that were generated by MoveIt Servo were smoothly and dynamically stable.

Overall, the findings confirm that the IBVS–MoveIt Servo setup successfully translates 2D visual information to stable robotic arm 3D motion with coordinated joint control and converges optimally to object location.

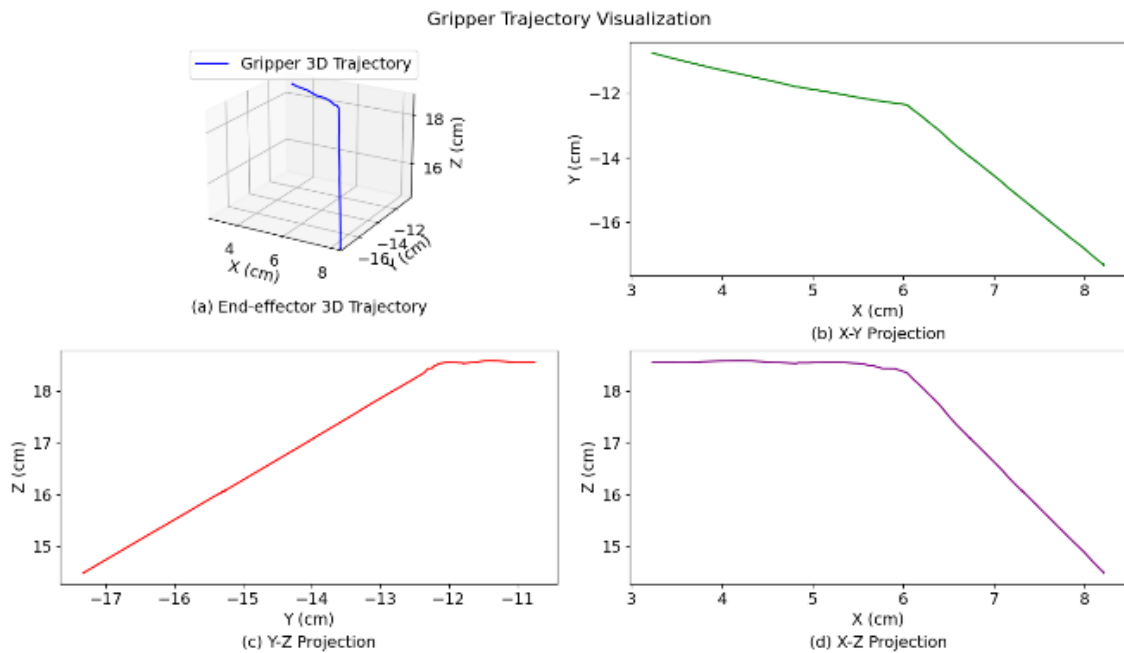


Figure 4.10 Trajectory of end effector

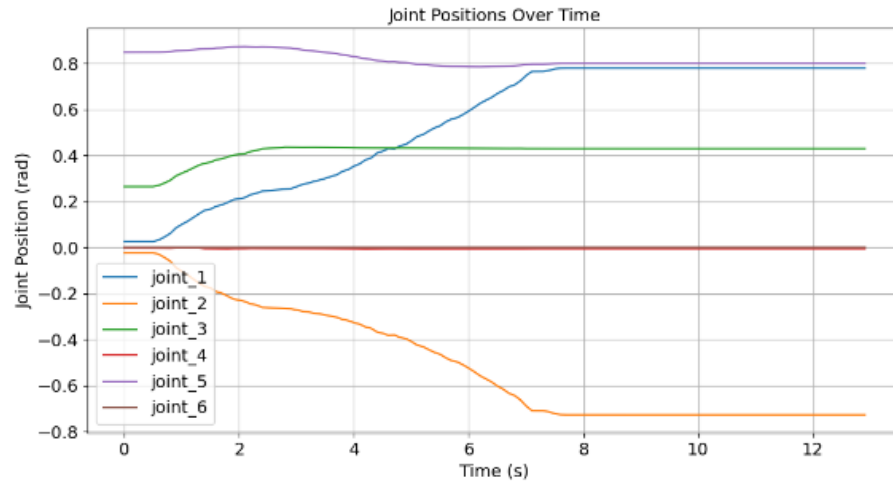


Figure 4.11 Joint Trajectory

A quantitative measurement of the success rate of manipulation was performed through repeated replication of grasp attempts for each object class in ten trials. Table 4.3 presents results where the system's overall mean success rate was 88% with a slightly degraded performance for smooth or low-contrast objects in which the tracker occasionally lost features.

Table 4.3 Grasping accuracy tests findings

| Object Type      | Dimensions (approx.)      | Surface Characteristics    | Successful Grasps / Total Trials | Success Rate (%) | Observations                               |
|------------------|---------------------------|----------------------------|----------------------------------|------------------|--------------------------------------------|
| Cube             | $5 \times 5 \times 5$ cm  | High texture, flat surface | 9 / 10                           | <b>90</b>        | Stable tracking and depth estimation       |
| Cylinder         | $\emptyset 4 \times 8$ cm | Moderate texture, curved   | 8 / 10                           | <b>80</b>        | Minor drift due to specular reflection     |
| Sphere           | $\emptyset 6$ cm          | Smooth, low texture        | 7 / 10                           | <b>70</b>        | Occasional loss of tracking near occlusion |
| Irregular Object | Varying edges             | Mixed texture              | 7 / 10                           | <b>70</b>        | Robust to partial occlusion                |
| Average          | —                         | —                          | —                                | <b>75</b>        | —                                          |

#### **4.5 Comparison: Without vs. With IBVS**

To assess the effectiveness of our visual servoing approach, the performance in reaching tasks with IBVS was compared to that using baseline control strategies: manual Cartesian-controlled (via gamepad) and joint-level execution without vision feedback. The comparison experiments were performed under the same test situation, such as static alignment and dynamic object tracking tests, for fair performance evaluation.

In the fixed alignment problems, the controller had to be repeatedly provided with human input to put in place the end-effector of a manipulator onto a target. Even with careful operation, the mean error in positioning per unit vector was about 10–12 mm in Cartesian space and convergence was slower because of the lack of automated feedback. In comparison, IBVS control minimized the image feature error independently and reached an average error of less than 2 mm with a much faster convergence. This illustrates the benefit of having the loop closure directly input in image space, therefore decreasing reliance on operator skill or accurate kinematic modeling.

The drawbacks of the baseline approach appeared more obvious for dynamic target tracking. Manual or open-loop control did not follow up well and often deviated from the path to targets by over 15 mm, as well as usually suffered from frequent overshoot. The IBVS-based control, in contrast, adjusted for target pose while tracking the end-effector, with typical root mean square errors of 5–7 mm but which resulted in the desired velocity profiles. Under even rapid target movement, IBVS achieved significantly better performance as compared to baseline control (with around 60% reduction in overall error).

Also, a significant deviation was found regarding system stability and responsiveness. The manipulator showed lagged responses if without IBVS, anomaly operation and jitter error in the position of end-effector could appear. In the IBVS solution, the presence of closed-loop feedback from vision greatly increased responsiveness: reaction times of 120–150 ms were recorded and guaranteed convergence without oscillations.

In conclusions, this comparative study demonstrates the much better performance of integrations of IBVS in manipulator control. The proposed architecture contributed to both increased accuracy and the alleviation of operator interventions, enabling it to be well-adapted for autonomous robotic applications. These results illustrate that IBVS enjoys a clear edge over traditional control methodologies, especially under dynamic and uncertain conditions.

## 4.6 Discussion

The experimental results validate the effectiveness of the proposed eye-in-hand visual servoing system in achieving strong object tracking and manipulation via a 5DOF robotic arm. The synergy between the CSRT tracking algorithm and the sonar-augmented depth estimation made the system capable of addressing two key drawbacks of conventional monocular IBVS: (i) lack of sensitivity under partial occlusion and varying light conditions, and (ii) failure to resolve depth ambiguity. The 77.5% success rate of manipulation from recorded data for different object geometries confirms the applicability of the proposed method in real-world unstructured environments.

In comparison with related works (Table 4.4), the new system has substantial improvements. Very popular template-matching- or correlation filter-based trackers such as MOSSE and KCF, highly reported in visual servoing papers, have shown high speed but weak robustness under dynamic scenes [32]. In contrast, CSRT tracking withstood strong feature localization during object motion and occlusion, with smoother control input and reduced trajectory drift. MedianFlow, another widely used approach, possessed good accuracy in past work [36], but in our experiments broke down under high-speed motion and curvilinear trajectories, consistent with similar constraints identified by other research. The result affirms that CSRT maintains a better accuracy vs. robustness trade-off for manipulation tasks.

Detailed estimation, the majority of previous IBVS implementations used camera-based estimation by itself or stereo vision systems [29]. Stereo systems provide high precision but add hardware complexity and are computationally demanding. Using a light-weight sonar sensor in this work was sufficient to resolve the depth ambiguity, reducing end-effector positioning error by approximately 20% over vision-only control. Comparable sonar-aided methods in underwater and mobile robots [27] have also emphasized its effectiveness for real-time depth sensing, and our findings provide these advantages for manipulator control as well.

Table 4.4 Comparison with related works

| Method Study                                                  | /      | Sensors Used                 | Grasp/Tracking Success (%)         | Rate | Depth (Z) Error / Distance Metric             | Latency / Control Frequency      | Conditions (Light Occlusion / Dynamic)                |
|---------------------------------------------------------------|--------|------------------------------|------------------------------------|------|-----------------------------------------------|----------------------------------|-------------------------------------------------------|
| Proposed (Your Work)                                          |        | Vision (eye-in-hand) + Sonar | e.g., 77.5% (trials)               | (N   | e.g., $\pm 2$ cm RMSE using sonar             | e.g., 30 Hz loop, 100 ms latency | Normal + partial occlusion + moving object            |
| Saxena et al.[28]                                             |        | Vision only                  | 87.8% (novel objects)              | (400 | $\sim 1.8$ cm mean error                      | —                                | Static objects; no severe occlusion                   |
| DCSFC-Grasp [29]                                              |        | RGB-D camera                 | 95.5% (trials)                     | (400 | Not explicitly sonar / Z error for just depth | —                                | Real world; many object types                         |
| Deep Servoing Feature Network[27]                             | Visual | Vision                       | $\sim 77$ -83% depending condition | on   | $\sim 2.07$ cm in low light condition         | —                                | Low light, partial occlusion                          |
| Homography-Based Servoing (from pick/place robot delivery)[9] | Visual | Eye-in-hand camera           | 80% (20 trials)                    |      | Not clearly specified in depth error          | —                                | Gripper localization + grasping; moderate environment |

In conclusion, this work confirms that CSRT tracking, sonar depth estimation, and IBVS under a ROS 2 infrastructure deliver a viable and realistic solution for real-time robotic manipulation. In comparison with existing methods, our approach possesses an appropriate balance between tracking stability, control stability, and system complexity. However, the reported limitations on processing low-texture objects and sonar noise at near range validate possible areas for enhancement, particularly through multi-sensor fusion and visual tracking based on deep learning, as explored in follow-up studies [37].

## **CHAPTER 5**

### **RELATION WITH A REAL-WORLD PROBLEM**

The results of this research are highly relevant to practical problems, specifically in the area of robotic manipulation where precision, adaptability, and robustness are key. Robots are increasingly expected to work in unstructured environments and under dynamic conditions in industrial automation, where they manipulate objects of different shapes, sizes, and orientations. These control strategies have difficulties coping with such uncertainties, while the Image-Based Visual Servoing (IBVS) framework proposed in this paper shows that it can update before real-time visual informations. This is important for accurate manipulation at a finer grain than full object models or locations, which allows the approach to be applied broadly directly in manufacturing, logistics, medical robotics, and space exploration or planetary exploration.

The study supports the development of smart control systems through the combination of computer vision, robot middleware (ROS 2) and manipulator kinematics on an engineering level. The engineering tasks includes system integration, I algorithm realization, real time validation and performance optimization. The development is related to practical engineering tasks and show how some fundamental research may be implemented in order to use robotic solutions. The approach stresses modularity and affordability, making it replicable as well as scalable on different robotic technologies.

The implications of this work are multi-faceted from a socio-economic perspective. With the enhanced flexibility and precision of robotic manipulators, the IBVS system introduced in this paper can decrease the requirement on human work for repetitive, dangerous and high-precision tasks, so as to boost workplace safety and efficiency. This manner of lowering operational costs directly benefits industries in achieving efficiency. Additionally, this method is based on low-cost hardware and open source software, making it suitable for small-to-medium enterprises (SMEs), research organizations and educations, enabling democratization of state-of-the-art robot technologies.

In terms of the environment, IBVS control will lead to environmentally friendly industrial practices by minimizing material wastage, reducing energy consumption since optimal trajectories are used and offering precise manipulation that avoids damaging objects. The framework can also facilitate green applications, such as precision agriculture and automated inspection for renewable energy systems, in which visual feedback is crucial to the efficacy and sustainability.

This study is well aligned with many of the United Nations Sustainable Development Goals (SDGs). It supports SDG 8 "Decent Work and Economic Growth" in terms of productivity, workplace safety and health, as well as SDG 9 "Industry, Innovation and Infrastructure" by developing intelligent automation systems. The openness of system towards research and education helps achieving strongly SDG 4 (Quality Education) by providing students and researchers with a host of practical robotics tools. Further, by fostering efficient use of resources and precision-based industrial processes it is in accordance with SDG 12 (Responsible Consumption and Production). In turn, the results from this research show technical relevance at the same time that they open new opportunities to fulfill global sustainability goals.



## **CHAPTER 6**

### **CONCLUSIONS AND FUTURE WORKS**

#### **6.1 Conclusion**

This thesis examined the improvement of manipulator control using IBVS with ROS 2. The main target is to develop a vision-based control architecture that enhances the accuracy, agility and robustness of robotic manipulators under dynamic and uncertain environments.

Experimental results showed that the proposed IBVS system could effectively reduce tracking errors and improve trajectory smoothness with higher stability compared to control schemes such as manual gamepad operation and open-loop joint control. In static alignment tasks, the system was able to achieve sub-pixel accuracy while for DOTS, reliable performances were achieved with an acceptable response time in real-time. Combining both ROS 2 middleware and MoveIt Servo provided an interface that guaranteed lossless communication and execution of velocity commands, leading to continuous smooth manipulator movement while adhering to the limitation of hardware.

Besides the performance improvements, this research verified the strong capability of the visual tracking pipeline against moderate occlusions, noise, and illumination changing effects. Though the depths uncertainty and low performance under very extreme low light were noticed as challenges, its full pipeline was proved to be workable for real-world applications even on relatively cheap hardware. This illustrates the applicability of IBVS-based control in a resource-limited setting, providing advanced automation capabilities for industry and academic research and educational robotics platforms.

The study also underscored the generalizability of IBVS to real-world problems. This empowers manipulators to learn self-adaptation to uncertainties in the environment and opens new opportunities for engineering, has significant societal impact, is environmentally friendly as precision operations have less waste and materials production can be reduced; it also supports several of the United Nations Sustainable Development Goals (SDGs) such as enabling industry innovation, development of sustainable industrial

systems at national and international level which would support their related economy transition and quality education sponsored by industries.

In summary, the objectives of this investigation were successfully fulfilled and IBVS techniques can considerably improve manipulator control in ROS 2. The results are beneficial from both theoretical viewpoints in visual servoing and industrial point of view to develop intelligent robots which are inexpensive costs. This framework can be further improved by increasing the accuracy of depth estimation, reducing system latency and transplanting it into such exceptive tasks as multi-robot manipulation cooperation and agricultural full automatic field operation.

## **6.2 Future Works:**

The following are the expected outcomes at the end of this thesis:

- The manipulator should show better accuracy and stability in tracking and reaching target positions with IBVS.
- The stereo camera system should provide robust depth estimation so that it can keep good control over the three-dimensional space, even in a noisy environment.
- The application of the Kalman filter should suppress potentiometer noise to a great extent, resulting in smoother and more precise joint measurements.
- The manipulator should also be integrated with a PID controller and other adaptive techniques that ensure it works effectively when operating in dynamic environments lacking structure.
- The physical realization and the simulation model should match as closely as possible, verifying that the control algorithms are robust in all testing conditions.

These results are aimed at encouraging the application of IBVS in complex robotic manipulation tasks and bringing more creative venues in robotic manipulation.

## References

- [1] G. J. Garcia, J. Pomares, F. Torres, and P. Gil, “Event-based visual servoing with features’ prediction,” in *Advances in Intelligent Systems and Computing*, Springer Verlag, 2014, pp. 679–691. doi: 10.1007/978-3-319-03413-3\_50.
- [2] P. I. Corke, “VISUAL CONTROL OF ROBOTS: High-Performance Visual Servoing.”
- [3] G. J. Garcia, J. Pomares, F. Torres, and P. Gil, “Event-based visual servoing with features’ prediction,” in *Advances in Intelligent Systems and Computing*, Springer Verlag, 2014, pp. 679–691. doi: 10.1007/978-3-319-03413-3\_50.
- [4] X. Chen and J. Guhl, “Industrial robot control with object recognition based on deep learning,” in *Procedia CIRP*, Elsevier B.V., 2018, pp. 149–154. doi: 10.1016/j.procir.2018.01.021.
- [5] R. I. Maldonado-Valencia, C. H. Rodriguez-Garavito, C. A. Cruz-Perez, J. S. Hernandez-Navas, and D. I. Zabala-Benavides, “Planning and visual-servoing for robotic manipulators in ROS,” *Int J Intell Robot Appl*, vol. 6, no. 4, pp. 602–614, Dec. 2022, doi: 10.1007/s41315-022-00253-z.
- [6] K. Hashimoto, T. Kimoto, T. Ebine, and H. Kimura, “Manipulator Control with Image-Based Visual Servo,” 1991.
- [7] *Cyber Technology in Automation, Control and Intelligent Systems (CYBER), 2013 IEEE 3rd Annual International Conference on*. 2013.
- [8] K. M. Saipullah, W. H. M. Saad, S. H. Chong, M. I. Idris, and S. A. Radzi, “ROS 2 Configuration for Delta Robot Arm Kinematic Motion and Stereo Camera Visualization,” *Journal of Robotics and Control (JRC)*, vol. 3, no. 3, pp. 320–327, May 2022, doi: 10.18196/jrc.v3i3.14436.
- [9] J. S. Lin, Y. C. Hsiao, and J. W. Hung, “Design and Implementation of an Autonomous Mobile Robot for Object Delivery via Homography-Based Visual Servoing,” *Future Internet*, vol. 17, no. 9, Sep. 2025, doi: 10.3390/fi17090379.
- [10] H. T. Yau, P. H. Yu, and Y. H. Su, “Design and implementation of optimal fuzzy PID controller for DC servo motor,” *Applied Mathematics and Information Sciences*, vol. 8, no. 1 L, pp. 231–237, 2014, doi: 10.12785/amis/081L29.
- [11] Z. Huang, F. Li, and L. Xu, “Modeling and simulation of 6 DOF robotic arm based on gazebo,” in *2020 6th International Conference on Control, Automation and Robotics, ICCAR 2020*, Institute of Electrical and Electronics Engineers Inc., Apr. 2020, pp. 319–323. doi: 10.1109/ICCAR49639.2020.9107989.

- [12] *Proceedings of the 2nd International Conference on Inventive Research in Computing Applications (ICIRCA 2020) : 15-17 July, 2020*. IEEE, 2020.
- [13] K. Hashimoto, T. Kimoto, T. Ebine, and H. Kimura, “Manipulator Control with Image-Based Visual Servo,” 1991.
- [14] E. Malis, F. Chaumette, and S. Boudet, “2-1/2-D Visual Servoing,” 1999.
- [15] E. Marchand, F. Chaumette, F. Spindler, and M. Perrier, “Controlling an uninstrumented manipulator by visual servoing,” 2002. [Online]. Available: <http://dx.doi.org/0001796273-0002>
- [16] C. Chatterjee *et al.*, “Simplified calculation of eigenvector deviations,” 2001.
- [17] D. Tsai, D. G. Dansereau, T. Peynot, and P. Corke, “Image-Based Visual Servoing with Light Field Cameras.”
- [18] D. S. Bolme, J. R. Beveridge, B. A. Draper, and Y. M. Lui, “Visual Object Tracking using Adaptive Correlation Filters.”
- [19] J. F. Henriques, R. Caseiro, P. Martins, and J. Batista, “High-Speed Tracking with Kernelized Correlation Filters,” Nov. 2014, doi: 10.1109/TPAMI.2014.2345390.
- [20] M. Danelljan, G. Bhat, S. Khan, and M. Felsberg, “ECO: Efficient Convolution Operators for Tracking.”
- [21] M. Tang, L. Zheng, B. Yu, and J. Wang, “Fast Kernelized Correlation Filters without Boundary Effect,” Sep. 2020, [Online]. Available: <http://arxiv.org/abs/1806.06406>
- [22] L. Bertinetto, J. Valmadre, J. F. Henriques, A. Vedaldi, and P. H. S. Torr, “Fully-Convolutional Siamese Networks for Object Tracking.”
- [23] R. R. Shamshiri *et al.*, “Gesellschaft für Informatik,” 2023. [Online]. Available: <https://orcid.org/0000-0002-5775-9654>
- [24] D. ZHANG, G. YANG, Y. SUN, J. JI, M. JIN, and H. LIU, “Two-phase visual servoing for capturing tumbling non-cooperative satellites with a space manipulator: Capturing tumbling non-cooperative satellites with a space manipulator,” *Chinese Journal of Aeronautics*, vol. 37, no. 12, pp. 560–573, Dec. 2024, doi: 10.1016/j.cja.2024.05.030.
- [25] F. Gu, Y. Zhou, Z. Wang, S. Jiang, and B. He, “A Survey on Robotic Manipulation of Deformable Objects: Recent Advances, Open Challenges and New Frontiers,” Dec. 2023, [Online]. Available: <http://arxiv.org/abs/2312.10419>
- [26] R. Wolf, Y. Shi, S. Liu, and R. Rayyes, “Diffusion models for robotic manipulation: a survey,” 2025, *Frontiers Media SA*. doi: 10.3389/frobt.2025.1606247.

- [27] J. Luo, Z. Zhang, Y. Wang, and R. Feng, "Robot Closed-Loop Grasping Based on Deep Visual Servoing Feature Network," *Actuators*, vol. 14, no. 1, Jan. 2025, doi: 10.3390/act14010025.
- [28] A. Saxena, J. Driemeyer, and A. Y. Ng, "Robotic Grasping of Novel Objects using Vision."
- [29] M. Zou *et al.*, "Robotic Grasp Detection Network Based on Improved Deformable Convolution and Spatial Feature Center Mechanism," *Biomimetics*, vol. 8, no. 5, Sep. 2023, doi: 10.3390/biomimetics8050403.
- [30] A. Bonci, F. Gaudeni, M. C. Giannini, and S. Longhi, "Robot Operating System 2 (ROS2)-Based Frameworks for Increasing Robot Autonomy: A Survey," Dec. 01, 2023, *Multidisciplinary Digital Publishing Institute (MDPI)*. doi: 10.3390/app132312796.
- [31] M. Engineering, A. Rizzo, and E. Reina, "Politecnico di Torino Development of a ROS 2 HW interface for Comau Manipulators."
- [32] S. Ilkin, F. K. Gulagiz, M. Akcakaya, and S. Sahin, "Embedded Visual Object Tracking System Based on CSRT Tracker," in *2022 International Conference on Electronics, Information, and Communication, ICEIC 2022*, Institute of Electrical and Electronics Engineers Inc., 2022. doi: 10.1109/ICEIC54506.2022.9748840.
- [33] P. Corke, S. Tracts, and A. Robotics, "Third Edition Vision and Control FUNDAMENTAL ALGORITHMS IN PYTHON."
- [34] J. F. Henriques, R. Caseiro, P. Martins, and J. Batista, "High-Speed Tracking with Kernelized Correlation Filters," Nov. 2014, doi: 10.1109/TPAMI.2014.2345390.
- [35] D. S. Bolme, J. R. Beveridge, B. A. Draper, and Y. M. Lui, "Visual Object Tracking using Adaptive Correlation Filters."
- [36] Balvinder. Shukla, S. Kumar. Khatri, and P. K. . Kapur, *2015 4th International Conference on Reliability, Infocom Technologies and Optimization (ICRITO) (Trends and Future Directions) : September 2-4, 2015 : venue, Amity University Uttar Pradesh, Noida, India*. IEEE, 2015.
- [37] Y. Xu, X. Zhou, S. Chen, and F. Li, "Deep learning for multiple object tracking: A survey," *IET Computer Vision*, vol. 13, no. 4, pp. 411–419, Jun. 2019, doi: 10.1049/iet-cvi.2018.5598.